# C++ BASIC PROGRAMMING ASSIGNMENT


## NAME: ANURAG VILAS DHONDGE

## COLLEGE: C-DAC {IACSD}

## DEPT: PGCP-AC

## ROLL: 268013

## PRN: 260841220030


DAY 1

```
/*
   1.  Write a program to accept an integer and
       determine whether it is positive, negative, or zero
       using if-else.
*/

#include <iostream>
using namespace std;

void Check_Number(float no)
{
   if(no > 0)
   {
      cout << "The integer is positive" << endl;
   }

   else if(no < 0)
   {
      cout << "->The integer is negative" << endl;
   }

   else if(no = 0)
   {
      cout << "The integer is zero" << endl;
   }

   else
   {
      cout << "Inalid Input";
```

```
    }

}

int main()
{
    float num = 0.0f , ret = 0.0f;

    cout << "Enter a integer : ";
    cin >> num;

    Check_Number(num);

    return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 1.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a integer : -1
->The integer is negative

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a integer : 0
The integer is zero

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a integer : 1
The integer is positive
```

```
/*  2.  Write a program to accept an integer and
        check whether the number is even or odd.
*/

#include <iostream>
using namespace std;

void Check_EvenOdd(int no)
{
   if(no % 2 == 0)
   {
      cout << "The number is Even";
   }

   else
   {
      cout << "The number is Odd";
   }

}

int main()
{
   int num;
   cout << "Enter number: ";
   cin >> num;

   if(num >= 0)
   {
      Check_EvenOdd(num);
   }

   else
   {
      cout << "Invalid Input";
   }

   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 0
The number is Even
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 1
The number is Odd
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>
```

```
/*
   3.  Write a program to accept two integers and
       display the larger number.
       Also handle the case when both numbers are equal.
*/
#include <iostream>
using namespace std;

void Display_Largest(int a, int b)
{
   if(a > b)
   {
      cout << "The Largest integer among two number = " << a;
   }

   else if(b > a)
   {
      cout << "The Largest integer among two number = " << b;
   }

   else if(a = b)
   {
      cout << "Both numbers are equal";
   }

   else
   {
      cout << "Invalid Input";
   }
}

int main()
{
   int a, b;
   cout << "Enter two integers: ";
   cin >> a >> b;

   Display_Largest(a, b);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 3.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter two integers: 10
20
The Largest integer among two number = 20
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter two integers: 10
10
Both numbers are equal
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter two integers: 20
10
The Largest integer among two number = 20
```

```
/*
   4.  Write a program to accept three numbers and
       display the largest number using if-else statements.
*/

#include <iostream>
using namespace std;

void Check_Largest(int a, int b, int c)
{
   if (a >= b && a >= c)
   {
      cout << "The largest number among three numbers = " << a;
   }

   else if (b >= a && b >= c)
   {
      cout << "The largest number among three numbers = " << b;
   }

   else if (c >= a && c >= b)
   {
      cout << "The largest number among three numbers = " << c;
   }

   else
   {
      cout << "Invalid Input";
   }
}

int main()
{
   int a, b, c;
   cout << "Enter three numbers: ";
   cin >> a >> b >> c;

   Check_Largest(a, b, c);
   return 0;
```

```
}
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 4.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter three numbers: 10
20
30
The largest number among three numbers = 30
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter three numbers: 10
10
10
The largest number among three numbers = 10
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter three numbers: 30
20
10
The largest number among three numbers = 30
```

```
/*
   5.  Accept marks obtained by a student and
       display the grade using the following criteria:
       90–100: A, 75–89: B,60–74: C, 40–59: D, below 40: Fail.
       Validate marks outside 0–100.
*/
#include <iostream>
using namespace std;

void Display_Grade(float marks)
{
   if (marks < 0 || marks > 100)
   {
      cout << "Invalid marks";
   }

   else if (marks >= 90)
   {
      cout << "The student got Grade A";
   }

   else if (marks >= 75 && marks < 90)
   {
      cout << "The student got Grade B";
   }

   else if (marks >= 60 && marks < 75)
   {
      cout << "The student got Grade C";
   }

   else if (marks >= 40 && marks < 60)
   {
      cout << "The student got Grade D";
   }

   else
   {
      cout << "Fail";
   }
}

int main()
{
   float marks;
   cout << "Enter students marks: ";
```

```
    cin >> marks;

    Display_Grade(marks);
    return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 5.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter students marks: -10
Invalid marks
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter students marks: 101
Invalid marks
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter students marks: 98
The student got Grade A
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter students marks: 80
The student got Grade B
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter students marks: 65
The student got Grade C
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter students marks: 45
The student got Grade D
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter students marks: 35
Fail
```

```
/*
   6.  Write a program to accept a year and
       determine whether it is a leap year.
*/

#include <iostream>
using namespace std;

void Check_LeapYear(int year)
{
   if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
   {
      cout << "The Year is Leap Year";
   }

   else
   {
      cout << "The Year is Not a Leap Year";
   }
}

int main()
{
   int year;
   cout << "Enter the year: ";
   cin >> year;

   Check_LeapYear(year);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 6.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter the year: 1000
The Year is Not a Leap Year
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter the year: 2000
The Year is Leap Year
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter the year: 2026
The Year is Not a Leap Year
```

```
/*
   7.  Accept the number of electricity units consumed and
       calculate the bill using slab rates of your choice.
       Display the total bill and explain the slabs used.
*/
#include <iostream>
using namespace std;

void Total_Bill(int units)
{
   float bill;

   if (units <= 100)
   {
      bill = units * 5;
   }

   else if (units <= 200 && units > 100)
   {
      bill = units * 6;
   }

   else
   {
      bill = units * 7;
   }

   cout << "Total Bill = Rs. " << bill << "\n As slab 1 is under 100 units for 5 rs per unit, \n"
<<" slab 2 is between 100 to 200 for 6 rs per unit,\n" << " and slab 3 is above 200 units for 7 rs
per unit.";
}

int main()
{
   int units;
   cout << "Enter electricity units consumed: ";
   cin >> units;

   if (units < 0)
   {
      cout << "Invalid units";
   }

   else
   {
      Total_Bill(units);
```

```
    }

    return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 7.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter electricity units consumed: 45
Total Bill = Rs. 225
 As slab 1 is under 100 units for 5 rs per unit,
 slab 2 is between 100 to 200 for 6 rs per unit,
 and slab 3 is above 200 units for 7 rs per unit.
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter electricity units consumed: 150
Total Bill = Rs. 900
 As slab 1 is under 100 units for 5 rs per unit,
 slab 2 is between 100 to 200 for 6 rs per unit,
 and slab 3 is above 200 units for 7 rs per unit.
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter electricity units consumed: 500
Total Bill = Rs. 3500
 As slab 1 is under 100 units for 5 rs per unit,
 slab 2 is between 100 to 200 for 6 rs per unit,
 and slab 3 is above 200 units for 7 rs per unit.
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>
```

```
/*
   8. Create a menu-driven calculator using switch-case.
      The user should select addition, subtraction, multiplication,
      division, or modulus and enter two numbers.
*/

#include <iostream>
using namespace std;

void Calculator(int choice, int a, int b)
{
   char wish;

   do
   {
      switch(choice)
      {
         case 1:
            cout << "Addition = " << a + b;
            break;

         case 2:
            cout << "Subtraction = " << a - b;
            break;

         case 3:
            cout << "Multiplication = " << a * b;
            break;

         case 4:
            if(b != 0)
            {
               cout << "Division = " << (float)a / b;
            }
            else
            {
               cout << "Cannot divide by zero";
            }
            break;

         case 5:
            if(b != 0)
            {
               cout << "Modulus = " << a % b;
            }
            else
```

```
            {
               cout << "Cannot divide by zero";
            }
            break;

         default:
            cout << "Invalid choice";
            break;
      }

      cout << "\nDo you wish to continue? (Y/N): ";
      cin >> wish;

      if(wish == 'y' || wish == 'Y')
      {
         cout << "\n1. Addition\n";
         cout << "2. Subtraction\n";
         cout << "3. Multiplication\n";
         cout << "4. Division\n";
         cout << "5. Modulus\n";

         cout << "Enter choice: ";
         cin >> choice;

         cout << "Enter two numbers: ";
         cin >> a >> b;
      }

   } while(wish == 'y' || wish == 'Y');
}

int main()
{
   int choice, a, b;

   cout << "1. Addition\n";
   cout << "2. Subtraction\n";
   cout << "3. Multiplication\n";
   cout << "4. Division\n";
   cout << "5. Modulus\n";

   cout << "Enter choice: ";
   cin >> choice;

   cout << "Enter two numbers: ";
   cin >> a >> b;
```

```
    Calculator(choice, a, b);

    return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 8.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
Enter choice: 1
Enter two numbers: 10
20
Addition = 30
Do you wish to continue? (Y/N): y

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
Enter choice: 2
Enter two numbers: 10
20
Subtraction = -10
Do you wish to continue? (Y/N): y
Do you wish to continue? (Y/N): y

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
Enter choice: 3
Enter two numbers: 10
20
Multiplication = 200
Do you wish to continue? (Y/N): y

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
Enter choice: 4
Enter two numbers: 10
20
Division = 0.5
Do you wish to continue? (Y/N): y
```

```
/*
    9.  Accept a number from 1 to 12 and
        display the corresponding month name using switch-case.
*/
#include <iostream>
using namespace std;

void Month_Name(int month)
{
    switch (month)
    {
        case 1:
            cout << "The month is January";
            break;
```

```
        case 2:
            cout << "The month is February";
            break;
        case 3:
            cout << "The month is March";
            break;
        case 4:
            cout << "The month is April";
            break;
        case 5:
            cout << "The month is May";
            break;
        case 6:
            cout << "The month is June";
            break;
        case 7:
            cout << "The month is July";
            break;
        case 8:
            cout << "The month is August";
            break;
        case 9:
            cout << "The month is September";
            break;
        case 10:
            cout << "The month is October";
            break;
        case 11:
            cout << "The month is November";
            break;
        case 12:
            cout << "The month is December";
            break;
        default:
            cout << "Invalid month";
    }
}

int main()
{
    int month;
    cout << "Enter month number: ";
    cin >> month;

    Month_Name(month);
    return 0;
```

```
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 9.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter month number: 1
The month is January
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter month number: 6
The month is June
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter month number: 12
The month is December
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>
```

```
/*
   10. Accept a character and use switch-case to
      determine whether it is a vowel.
      If it is not a vowel, display that it is a consonant.
*/

#include <iostream>
using namespace std;

void Check_Vowel(char ch)
{
   char wish;
   do
```

```
    {
        switch(ch)
        {
            case 'a':
            case 'e':
            case 'i':
            case 'o':
            case 'u':
            case 'A':
            case 'E':
            case 'I':
            case 'O':
            case 'U':
                cout << "The character " << ch << " is vowel";
                break;

            default:
                cout << "The character " << ch << " is consonant";
                break;
        }

        cout << "\nDo you wish to continue? (Y/N): ";
        cin >> wish;

    } while(wish == 'y' || wish == 'Y');
}

int main()
{
    char ch;
    cout << "Enter a character: ";
    cin >> ch;
    Check_Vowel(ch);

    return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 10.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a character: a
The character a is vowel
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a character: e
The character e is vowel
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a character: i
The character i is vowel
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a character: w
The character w is consonant
```

```
/*
   11. Create a menu using switch-case to calculate the
       area of a circle, rectangle, square, or triangle
       based on the user's choice.
*/

#include <iostream>
using namespace std;

void Area(int choice)
{
   float r, l, b, s, h;
```

```
char wish;

do
{
   switch(choice)
   {
      case 1:
         cout << "Enter radius: ";
         cin >> r;
         cout << "Area = " << 3.14 * r * r;
         break;

      case 2:
         cout << "Enter length and breadth: ";
         cin >> l >> b;
         cout << "Area = " << l * b;
         break;

      case 3:
         cout << "Enter side: ";
         cin >> s;
         cout << "Area = " << s * s;
         break;

      case 4:
         cout << "Enter base and height: ";
         cin >> b >> h;
         cout << "Area = " << 0.5 * b * h;
         break;

      default:
         cout << "Invalid choice";
         break;
   }

   cout << "\nDo you wish to continue? (Y/N): ";
   cin >> wish;

   if(wish == 'y' || wish == 'Y')
   {
      cout << "\n1. Circle\n";
      cout << "2. Rectangle\n";
      cout << "3. Square\n";
      cout << "4. Triangle\n";

      cout << "Enter choice: ";
```

```
            cin >> choice;
        }

    } while(wish == 'y' || wish == 'Y');
}

int main()
{
    int choice;

    cout << "1. Circle\n";
    cout << "2. Rectangle\n";
    cout << "3. Square\n";
    cout << "4. Triangle\n";

    cout << "Enter choice: ";
    cin >> choice;

    Area(choice);

    return 0;
```

```
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 11.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
1. Circle
2. Rectangle
3. Square
4. Triangle
Enter choice: 1
Enter radius: 10
Area = 314
Do you wish to continue? (Y/N): y

1. Circle
2. Rectangle
3. Square
4. Triangle
Enter choice: 2
Enter length and breadth: 10
20
Area = 200
Do you wish to continue? (Y/N): y

1. Circle
2. Rectangle
3. Square
4. Triangle
Enter choice: 3
Enter side: 10
Area = 100
```

```
Do you wish to continue? (Y/N): y

1. Circle
2. Rectangle
3. Square
4. Triangle
Enter choice: 4
Enter base and height: 10
20
Area = 100
Do you wish to continue? (Y/N): n

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>
```

```
/*
   12. Create a simple ATM menu using switch-case with options
      such as Check Balance, Deposit, Withdraw, and Exit.
      Maintain the balance during the program execution.
*/

#include <iostream>
using namespace std;

void ATM(int choice, float &balance)
{
   float amount;
   char wish;
   do
   {
      switch(choice)
      {
         case 1:
            cout << "\nBalance = " << balance;
            break;

         case 2:
            cout << "\nEnter deposit amount: ";
            cin >> amount;

            if(amount > 0)
            {
               balance = balance + amount;
               cout << "Amount deposited successfully";
               cout << "\nNew Balance = " << balance;
            }

            else
            {
               cout << "Invalid amount";
            }

            break;

         case 3:
            cout << "\nEnter withdrawal amount: ";
            cin >> amount;

            if(amount > 0 && amount <= balance)
            {
               balance = balance - amount;
```

```
            cout << "Amount withdrawn successfully";
            cout << "\nNew Balance = " << balance;
        }

        else
        {
            cout << "Insufficient balance or invalid amount";
        }

        break;

    case 4:
        cout << "\nThank you for using ATM";
        return;

    default:
        cout << "\nInvalid choice";
        break;
    }

    cout << "\n\nDo you wish to continue? (Y/N): ";
    cin >> wish;

    if(wish == 'y' || wish == 'Y')
    {
        cout << "\n1. Check Balance";
        cout << "\n2. Deposit";
        cout << "\n3. Withdraw";
        cout << "\n4. Exit";

        cout << "\nEnter choice: ";
        cin >> choice;
    }

  } while(wish == 'y' || wish == 'Y');
}

int main()
{
  int choice;
  float balance = 1000;

  cout << "*****ATM MENU*****";
  cout << "\n1. Check Balance";
  cout << "\n2. Deposit";
  cout << "\n3. Withdraw";
```

```
    cout << "\n4. Exit";

    cout << "\nEnter choice: ";
    cin >> choice;

    ATM(choice, balance);

    return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 12.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
*****ATM MENU*****
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 1

Balance = 1000

Do you wish to continue? (Y/N): y

1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 2

Enter deposit amount: 10000
Amount deposited successfully
New Balance = 11000

Do you wish to continue? (Y/N): y
```

```
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 3

Enter withdrawal amount: 500
Amount withdrawn successfully
New Balance = 10500

Do you wish to continue? (Y/N): y

1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 4

Thank you for using ATM
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>
```

```
/*
    13. Write programs using for, while, and do-while loops to print numbers from 1 to N.
*/

#include <iostream>
using namespace std;

void Print_Numbers(int n)
{
   int i;

   cout << "Using for     : ";
   for(i = 1; i <= n; i++)
   {
      cout << i << " ";
   }

   cout << "\nUsing while   : ";
   i = 1;
   while(i <= n)
   {
      cout << i << " ";
      i++;
   }

   cout << "\nUsing do-while: ";
   i = 1;
   do
   {
      cout << i << " ";
      i++;
   } while(i <= n);
}

int main()
{
   int n;
   cout << "Enter number  : ";
   cin >> n;

   Print_Numbers(n);
   return 0;
```

```
}
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 13.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number  : 10
Using for     : 1 2 3 4 5 6 7 8 9 10
Using while   : 1 2 3 4 5 6 7 8 9 10
Using do-while: 1 2 3 4 5 6 7 8 9 10
```

```
/*
   14. Accept N and calculate the sum of the first N natural numbers using a loop.
*/

#include <iostream>
using namespace std;

void Sum_Natural(int n)
{
   int sum = 0;

   for(int i = 1; i <= n; i++)
      sum = sum + i;

   cout << "The sum of the first " << n <<" natural numbers = " << sum;
}

int main()
{
   int n;
   cout << "Enter Number : ";
   cin >> n;

   Sum_Natural(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 14.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter Number : 10
The sum of the first 10 natural numbers = 55
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter Number : 20
The sum of the first 20 natural numbers = 210
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter Number : 30
The sum of the first 30 natural numbers = 465
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>
```

```
/*
   15. Accept a number and print its multiplication table from 1 to 10 using a loop.
*/

#include <iostream>
using namespace std;

void Table(int n)
{
   for(int i = 1; i <= 10; i++)
   {
      cout << n << " * " << i << " = " << n * i << endl;
   }
}

int main()
{
   int n;
   cout << "Enter number: ";
   cin >> n;

   Table(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 15.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 10
10 * 1 = 10
10 * 2 = 20
10 * 3 = 30
10 * 4 = 40
10 * 5 = 50
10 * 6 = 60
10 * 7 = 70
10 * 8 = 80
10 * 9 = 90
10 * 10 = 100

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 5
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

```
/*
   16. Accept a positive integer and calculate its factorial using a loop.
*/

#include <iostream>
using namespace std;

void Factorial(int n)
{
   int fact = 1;

   for(int i = 1; i <= n; i++)
      fact = fact * i;

   cout << "The Factorial of " << n << " number is = " << fact;
}

int main()
{
   int n;
   cout << "Enter number: ";
   cin >> n;

   Factorial(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 16.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 10
The Factorial of 10 number is = 3628800
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 20
The Factorial of 20 number is = -2102132736
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 5
The Factorial of 5 number is = 120
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 4
The Factorial of 4 number is = 24
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 2
The Factorial of 2 number is = 2
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 1
The Factorial of 1 number is = 1
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter number: 0
The Factorial of 0 number is = 1
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>
```

```
/*
   17. Accept an integer and reverse its digits using a while loop.
*/

#include <iostream>
using namespace std;

void Reverse_Number(int n)
{
   int rev = 0;

   while(n != 0)
   {
      rev = rev * 10 + n % 10;
      n = n / 10;
   }

   cout << "Reverse = " << rev;
}

int main()
{
   int n;
   cout << "Enter a number: ";
   cin >> n;

   Reverse_Number(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 17.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 10
Reverse = 1
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 56
Reverse = 65
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 87
Reverse = 78
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 45321
Reverse = 12354
```

```
/*
   18. Accept an integer and determine whether it is a palindrome number.
*/

#include <iostream>
using namespace std;

void Palindrome(int n)
{
   int original = n;
   int rev = 0;

   while(n != 0)
   {
      rev = rev * 10 + n % 10;
      n /= 10;
   }

   if(original == rev)
   {
      cout << "Palindrome";
   }

   else
   {
      cout << "Not Palindrome";
   }
}

int main()
{
   int n;
   cout << "Enter a number: ";
   cin >> n;

   Palindrome(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 18.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 10
Not Palindrome
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 131
Palindrome
```

```
/*
   19. Accept an integer and count the number of digits using a loop.
*/

#include <iostream>
using namespace std;

void Count_Digits(int n)
{
   int count = 0;

   do
   {
      count++;
      n = n / 10;
   } while(n != 0);

   cout << "Number of digits = " << count;
}

int main()
{
   int n;
   cout << "Enter a number: ";
   cin >> n;

   Count_Digits(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 19.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 10
Number of digits = 2
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 123
Number of digits = 3
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 0
Number of digits = 1
```

```
/*
   20. Accept an integer and calculate the sum of all its digits.
*/

#include <iostream>
using namespace std;

void Sum_Digits(int n)
{
   int sum = 0;

   while(n != 0)
   {
      sum = sum + (n % 10);
      n = n / 10;
   }

   cout << "Sum of digits = " << sum;
}

int main()
{
   int n;
   cout << "Enter a number: ";
   cin >> n;

   Sum_Digits(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>g++ 20.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 10
Sum of digits = 1
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 123
Sum of digits = 6
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 456
Sum of digits = 15
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 0000
Sum of digits = 0
```

```
/*
   21. Accept an integer and determine whether it is a prime number using a loop.
*/

#include <iostream>
using namespace std;

void Check_Prime(int n)
{
   int count = 0;

   for(int i = 1; i <= n; i++)
   {
      if(n % i == 0)
         count++;
   }

   if(count == 2)
   {
      cout << n << " is a Prime Number";
   }

   else
   {
      cout << n << " is a Not Prime Number";
   }

}

int main()
{
   int n;
   cout << "Enter a number: ";
   cin >> n;

   Check_Prime(n);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 10
10 is a Not Prime Number
D:\CDAC\ASSIGNMENTS\College_Assignments\1.CPP>myexe.exe
Enter a number: 11
11 is a Prime Number
```

# C++ BASIC PROGRAMMING ASSIGNMENT

## DAY 2

```
/*
   1.  Write a menu-driven C++ program to dynamically allocate an integer array of N elements
and perform
       the following operations using user-defined functions:
       1. Accept and display the array elements.
       2. Find and display the largest and smallest elements.
       3. Calculate and display the sum and average of the array elements.
       4. Count and display the number of even and odd elements.
       5. Search for a given element using linear search.
       6. Exit the program.
*/

#include <iostream>
using namespace std;

// 1.Accept array elements
void Accept(int *arr, int n)
{
   cout << "Enter " << n << " elements: ";

   for (int i = 0; i < n; i++)
   {
      cin >> arr[i];
   }
}

// 1.Display array elements
void Display(int *arr, int n)
{
   cout << "Array elements: ";

   for (int i = 0; i < n; i++)
   {
      cout << arr[i] << " ";
   }

   cout << endl;
}

// 2.Find largest and smallest elements
void Largest_Smallest(int *arr, int n)
{
```

```
    int largest = arr[0];
    int smallest = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (arr[i] > largest)
            largest = arr[i];

        if (arr[i] < smallest)
            smallest = arr[i];
    }

    cout << "Largest = " << largest << endl;
    cout << "Smallest = " << smallest << endl;
}

// 3.Calculate sum and average
void Sum_Average(int *arr, int n)
{
    int sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += arr[i];
    }

    float average = (float)sum / n;

    cout << "Sum = " << sum << endl;
    cout << "Average = " << average << endl;
}

// 4.Count even and odd elements
void Even_Odd(int *arr, int n)
{
    int even = 0;
    int odd = 0;

    for (int i = 0; i < n; i++)
    {
        if (arr[i] % 2 == 0)
            even++;
        else
            odd++;
    }
    cout << "Even elements = " << even << endl;
```

```
    cout << "Odd elements = " << odd << endl;
}

// 5.Linear search
void Linear_Search(int *arr, int n)
{
    int key;
    bool found = false;

    cout << "Enter element to search: ";
    cin >> key;

    for (int i = 0; i < n; i++)
    {
        if (arr[i] == key)
        {
            cout << "Element found at position " << i + 1 << endl;
            found = true;
            break;
        }
    }

    if (!found)
        cout << "Element not found." << endl;
}

int main()
{
    int n, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    // Dynamically allocate array
    int *arr = new int[n];

    do
    {
        cout << "\n----- MENU -----\n";
        cout << "1. Accept and Display Array\n";
        cout << "2. Largest and Smallest\n";
        cout << "3. Sum and Average\n";
        cout << "4. Count Even and Odd\n";
        cout << "5. Linear Search\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
```

```
    cin >> choice;

    switch (choice)
    {
    case 1:
        Accept(arr, n);
        Display(arr, n);
        break;

    case 2:
        Largest_Smallest(arr, n);
        break;

    case 3:
        Sum_Average(arr, n);
        break;

    case 4:
        Even_Odd(arr, n);
        break;

    case 5:
        Linear_Search(arr, n);
        break;

    case 6:
        cout << "Exiting program..."; //6.Exit
        break;

    default:
        cout << "Invalid choice!";
    }

} while (choice != 6);

// Release dynamically allocated memory
delete[] arr;

return 0;
```

```
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Dynamic Memory Allocation - Arrays>g++ 1.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Dynamic Memory Allocation - Arrays>myexe.exe
Enter number of elements: 5

----- MENU -----
1. Accept and Display Array
2. Largest and Smallest
3. Sum and Average
4. Count Even and Odd
5. Linear Search
6. Exit
Enter your choice: 1
Enter 5 elements: 10
20
30
40
50
Array elements: 10 20 30 40 50

----- MENU -----
1. Accept and Display Array
2. Largest and Smallest
3. Sum and Average
4. Count Even and Odd
5. Linear Search
6. Exit
Enter your choice: 2
Largest = 50
Smallest = 10

----- MENU -----
1. Accept and Display Array
2. Largest and Smallest
3. Sum and Average
4. Count Even and Odd
5. Linear Search
6. Exit
```

```
----- MENU -----
1. Accept and Display Array
2. Largest and Smallest
3. Sum and Average
4. Count Even and Odd
5. Linear Search
6. Exit
Enter your choice: 3
Sum = 150
Average = 30

----- MENU -----
1. Accept and Display Array
2. Largest and Smallest
3. Sum and Average
4. Count Even and Odd
5. Linear Search
6. Exit
Enter your choice: 4
Even elements = 5
Odd elements = 0

----- MENU -----
1. Accept and Display Array
2. Largest and Smallest
3. Sum and Average
4. Count Even and Odd
5. Linear Search
6. Exit
Enter your choice: 5
Enter element to search:

20
Element found at position 2
```

```
----- MENU -----
1. Accept and Display Array
2. Largest and Smallest
3. Sum and Average
4. Count Even and Odd
5. Linear Search
6. Exit
Enter your choice: 6
Exiting program...
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Dynamic Memory Allocation - Arrays>
```

```
/*
   2.  Write a menu-driven C++ program to dynamically allocate memory for the prices of N products in a
       store and perform the following operations using user-defined functions:
       1. Accept and display the product prices.
       2. Find and display the highest and lowest-priced products.
       3. Calculate and display the total inventory value and average product price.
       4. Count the number of products whose price is above and below the average price.
       5. Search for a particular product price using linear search.
       6. Apply a given discount percentage to all product prices and display the updated prices.
       7. Exit the program
*/

#include <iostream>
using namespace std;

// 1.Function to accept prices
void Accept_Prices(float *price, int n)
{
   cout << "Enter prices of " << n << " products:\n";

   for (int i = 0; i < n; i++)
   {
      cout << "Product " << i + 1 << ": ";
      cin >> price[i];
   }
}

// 1.Function to display prices
void Display_Prices(float *price, int n)
{
   cout << "\nProduct Prices:\n";

   for (int i = 0; i < n; i++)
   {
      cout << "Product " << i + 1 << ": " << price[i] << endl;
   }
}

// 2.Function to find highest and lowest price
void Find_HighLow(float *price, int n)
{
   float highest = price[0];
   float lowest = price[0];

   for (int i = 1; i < n; i++)
```

```
    {
        if (price[i] > highest)
            highest = price[i];

        if (price[i] < lowest)
            lowest = price[i];
    }

    cout << "\nHighest Price: " << highest << endl;
    cout << "Lowest Price: " << lowest << endl;
}

// 3.Function to calculate total and average
float Calculate_Average(float *price, int n)
{
    float total = 0;

    for (int i = 0; i < n; i++)
    {
        total += price[i];
    }

    float average = total / n;

    cout << "\nTotal Inventory Value: " << total << endl;
    cout << "Average Product Price: " << average << endl;

    return average;
}

// 4.Function to count prices above and below average
void Count_AboveBelow(float *price, int n, float average)
{
    int above = 0;
    int below = 0;

    for (int i = 0; i < n; i++)
    {
        if (price[i] > average)
            above++;
        else if (price[i] < average)
            below++;
    }

    cout << "\nProducts above average: " << above << endl;
    cout << "Products below average: " << below << endl;
```

```
}

// 5.Function for linear search
void Search_Price(float *price, int n)
{
   float search;
   bool found = false;

   cout << "\nEnter price to search: ";
   cin >> search;

   for (int i = 0; i < n; i++)
   {
      if (price[i] == search)
      {
         cout << "Price found at Product " << i + 1 << endl;
         found = true;
      }
   }

   if (!found)
      cout << "Price not found." << endl;
}

// 6.Function to apply discount
void Apply_Discount(float *price, int n)
{
   float discount;

   cout << "\nEnter discount percentage: ";
   cin >> discount;

   for (int i = 0; i < n; i++)
   {
      price[i] = price[i] - (price[i] * discount / 100);
   }

   cout << "\nPrices after discount:\n";
   Display_Prices(price, n);
}

int main()
{
   int n, choice;

   cout << "Enter number of products: ";
```

```
cin >> n;

// Dynamic memory allocation
float *price = new float[n];

Accept_Prices(price, n);

do
{
   cout << "\n*****MENU*****\n";
   cout << "1. Display Product Prices\n";
   cout << "2. Find Highest and Lowest Price\n";
   cout << "3. Calculate Total and Average\n";
   cout << "4. Count Above and Below Average\n";
   cout << "5. Search Product Price\n";
   cout << "6. Apply Discount\n";
   cout << "7. Exit\n";
   cout << "Enter your choice: ";
   cin >> choice;

   switch (choice)
   {
      case 1:
         Display_Prices(price, n);
         break;

      case 2:
         Find_HighLow(price, n);
         break;

      case 3:
         Calculate_Average(price, n);
         break;

      case 4:
      {
         float average = Calculate_Average(price, n);
         Count_AboveBelow(price, n, average);
         break;
      }

      case 5:
         Search_Price(price, n);
         break;

      case 6:
```

```
            Accept_Prices(price, n);
            break;

        case 7:
            cout << "\nExiting program...\n"; // 7. Exit
            break;

        default:
            cout << "\nInvalid choice! Please try again.\n";
      }

   } while (choice != 7);

   // Free dynamically allocated memory
   delete[] price;

   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Dynamic Memory Allocation - Arrays>g++ 2.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Dynamic Memory Allocation - Arrays>myexe.exe
Enter number of products: 5
Enter prices of 5 products:
Product 1: 10
Product 2: 20
Product 3: 30
Product 4: 40
Product 5: 50

*****MENU*****
1. Display Product Prices
2. Find Highest and Lowest Price
3. Calculate Total and Average
4. Count Above and Below Average
5. Search Product Price
6. Apply Discount
7. Exit
Enter your choice: 1

Product Prices:
Product 1: 10
Product 2: 20
Product 3: 30
Product 4: 40
Product 5: 50

*****MENU*****
1. Display Product Prices
2. Find Highest and Lowest Price
3. Calculate Total and Average
4. Count Above and Below Average
5. Search Product Price
6. Apply Discount
7. Exit
Enter your choice: 2

Highest Price: 50
```

```
Highest Price: 50
Lowest Price: 10

*****MENU*****
1. Display Product Prices
2. Find Highest and Lowest Price
3. Calculate Total and Average
4. Count Above and Below Average
5. Search Product Price
6. Apply Discount
7. Exit
Enter your choice: 3

Total Inventory Value: 150
Average Product Price: 30

*****MENU*****
1. Display Product Prices
2. Find Highest and Lowest Price
3. Calculate Total and Average
4. Count Above and Below Average
5. Search Product Price
6. Apply Discount
7. Exit
Enter your choice: 4

Total Inventory Value: 150
Average Product Price: 30

Products above average: 2
Products below average: 2

*****MENU*****
1. Display Product Prices
2. Find Highest and Lowest Price
3. Calculate Total and Average
4. Count Above and Below Average
5. Search Product Price
6. Apply Discount
7. Exit
```

```
*****MENU*****
1. Display Product Prices
2. Find Highest and Lowest Price
3. Calculate Total and Average
4. Count Above and Below Average
5. Search Product Price
6. Apply Discount
7. Exit
Enter your choice: 5

Enter price to search: 6
Price not found.

*****MENU*****
1. Display Product Prices
2. Find Highest and Lowest Price
3. Calculate Total and Average
4. Count Above and Below Average
5. Search Product Price
6. Apply Discount
7. Exit
Enter your choice: 7

Exiting program...

D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Dynamic Memory Allocation - Arrays>
```

```
/*
   3.  Write a function that accepts the addresses of two integers
       and swaps their values using pointers.
*/

#include <iostream>
using namespace std;

// Swap two values using pointers
void Swap(float *a, float *b)
{
   float temp = *a;
   *a = *b;
   *b = temp;

   cout << "After Swap the values are : \n"<< "a = " << *a << " and b = " << *b;
}

int main()
{
   float a,b;
   cout << "Enter the number to swap. \n";
   cin >> a >> b;

   // Pass the addresses of a and b to the function
   Swap(&a, &b);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>g++ 1.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>myexe.exe
Enter the number to swap.
10
20
After Swap the values are :
a = 20 and b = 10
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>
```

```
/*
   4.  Write a function that accepts two integers using pass by reference
       and swaps their values.
*/

#include <iostream>
using namespace std;

// Swap two values using pass by reference
void Swap(float &a, float &b)
{
   float temp = a;
   a = b;
   b = temp;

   cout << "After Swap the values are : \n"<< "a = " << a << " and b = " << b <<" using pass by
reference ";
}

int main()
{
   float a,b;
   cout << "Enter the number to swap. \n";
   cin >> a >> b;

   // Pass a and b by reference
   Swap(a, b);
   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>g++ 2.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>myexe.exe
Enter the number to swap.
20
30
After Swap the values are :
a = 30 and b = 20 using pass by reference
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>
```

```
/*
    5. Write a function using pass by reference to calculate the
       sum and difference of two numbers and return both results
       through reference parameters.
*/

#include <iostream>
using namespace std;

// Accept two numbers and return sum and difference through references
void Sum_Diff(float &a, float &b, float &sum, float &diff)
{
   sum = a + b;
   diff = a - b;
}

int main()
{
   float a, b, sum, diff;

   cout << "Enter two numbers: ";
   cin >> a >> b;

   Sum_Diff(a, b, sum, diff);  // Pass input and result variables by reference

   cout << "Sum of numbers is: " << sum << endl;
   cout << "Difference of numbers is: " << diff;

   return 0;
   }
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>myexe.exe
Enter two numbers: 10
20
Sum of numbers is: 30
Difference of numbers is: -10
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>
```

```
/*
```

```
    . Write a function using pass by reference to
      reverse the digits of a number and
      update the original variable in main.
*/

#include <iostream>
using namespace std;

// Reverse the digits and update the original number using reference
void Reverse(int &num)
{
   int reverse = 0;
   int rem;

   while (num != 0)
   {
      rem = num % 10;
      reverse = reverse * 10 + rem;
      num = num / 10;
   }

   num = reverse;
}

int main()
{
   int num;

   cout << "Enter a number: ";
   cin >> num;

   Reverse(num);  // Pass number by reference

   cout << "After reversing: " << num;

   return 0;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\2.CPP\Pass by Address and Reference>myexe.exe
Enter a number: 12
After reversing: 21
```

DAY 3

```
/*  1.  Bank Account
      Create a BankAccount class with account number, customer name, and balance. Use a
      constructor to initialize account details. Implement member functions to deposit, withdraw,
      and display the balance. Create multiple account objects using an array of objects.
*/

#pragma once
#include<iostream>
#include<string>
using namespace std;

class BankAccount
{
        private:
                int accountno;
                string customername;
                float balance;

        public:
                BankAccount()
                {
                        accountno = 0;
                        customername = "";
                        balance = 0;
```

```
}

BankAccount(int a, string n, float b)
{
        accountno = a;
        customername = n;
        balance = b;
}

void accept()
{
        cout<<"\n Enter account no: ";
        cin>>accountno;

        cout<<" Enter customer name: ";
        cin>>customername;

        cout<<" Enter balance: ";
        cin>>balance;
}

void deposit()
{
        float amount;
```

```
        cout<<"\n Enter amount to deposit: ";
        cin>>amount;

        balance = balance + amount;
}

void withdraw()
{
        float amount;

        cout<<"\n Enter amount to withdraw: ";
        cin>>amount;

        if(amount <= balance)
                balance = balance - amount;
        else
                cout<<"\n Insufficient balance";
}

void display()
{
        cout<<"\n Account No: "<<accountno;
        cout<<"\n Customer Name: "<<customername;
        cout<<"\n Balance: "<<balance;
}
```

```
};

/*  1.  Bank Account
      Create a BankAccount class with account number, customer name, and balance. Use a
      constructor to initialize account details. Implement member functions to deposit, withdraw,
      and display the balance. Create multiple account objects using an array of objects.
*/


#include"1.Bank_Account.h"

int main()
{
      int noa;

      cout<<"\n Enter number of accounts: ";
      cin>>noa;

      BankAccount* barr = new BankAccount[noa];

      for(int i = 0 ; i < noa ; i++)
      {
            cout<<"\n Enter details of account "<<i+1;
            barr[i].accept();
      }
```

```
    for(int i = 0 ; i < noa ; i++)
    {
        cout<<"\n\n Account "<< i + 1;
        barr[i].display();
    }

    int choice;

    cout<<"\n\n Enter account number/index for transaction: ";
    cin>>choice;

    barr[choice-1].deposit();

    cout<<"\n\n After deposit:";
    barr[choice-1].display();

    barr[choice-1].withdraw();

    cout<<"\n\n After withdrawal:";
    barr[choice-1].display();

    delete[] barr;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>g++ 1.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>myexe.exe

 Enter number of accounts: 2

 Enter details of account 1
 Enter account no: 101
 Enter customer name: abc
 Enter balance: 10000

 Enter details of account 2
 Enter account no: 102
 Enter customer name: def
 Enter balance: 20000


 Account 1
 Account No: 101
 Customer Name: abc
 Balance: 10000

 Account 2
 Account No: 102
 Customer Name: def
 Balance: 20000

 Enter account number/index for transaction: 2

 Enter amount to deposit: 1000


 After deposit:
 Account No: 102
 Customer Name: def
 Balance: 21000
 Enter amount to withdraw: 500


 After withdrawal:
 Account No: 102
 Customer Name: def
 Balance: 20500
```

```
/*  2.  Book Management
        Create a Book class with book ID, title, author, and price. Use a constructor to initialize the
        data members. Store details of multiple books in an array of objects and search for a book
        using its ID.
*/

#pragma once
#include<iostream>
#include<string>
using namespace std;

class Book
{
        private:
                int bookid;
                string title;
                string author;
                float price;

        public:
                Book()
                {
                        bookid = 0;
                        title = "";
                        author = "";
                        price = 0;
                }

                Book(int id, string t, string a, float p)
                {
                        bookid = id;
                        title = t;
```

```
        author = a;
        price = p;
}

void accept()
{
        cout<<"\n Enter book id: ";
        cin>>bookid;

        cout<<" Enter title: ";
        cin>>title;

        cout<<" Enter author: ";
        cin>>author;

        cout<<" Enter price: ";
        cin>>price;
}

void display()
{
        cout<<"\n Book ID: "<<bookid;
        cout<<"\n Title: "<<title;
        cout<<"\n Author: "<<author;
        cout<<"\n Price: "<<price;
}

bool search(int id)
{
        if(bookid == id)
                return true;
```

```
                return false;
        }
};



#include "2.Book_Management.h"

int main()
{
        int nob;

        cout<<"\n Enter number of books: ";
        cin>>nob;

        Book* barr = new Book[nob];

        for(int i=0; i<nob; i++)
        {
                cout<<"\n Enter details of book "<<i+1;
                barr[i].accept();
        }

        cout<<"\n\n All books----------";

        for(int i=0; i<nob; i++)
        {
                barr[i].display();
                cout<<endl;
        }

        int id;
```

```
        cout<<"\n Enter book id to search: ";
        cin>>id;

        for(int i=0; i<nob; i++)
        {
                if(barr[i].search(id))
                {
                        cout<<"\n Book found----------";
                        barr[i].display();
                        break;
                }
        }

        delete[] barr;
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>g++ 2.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>myexe.exe

 Enter number of books: 2

 Enter details of book 1
 Enter book id: 101
 Enter title: abc
 Enter author: xyz
 Enter price: 500

 Enter details of book 2
 Enter book id: 102
 Enter title: wer
 Enter author: tyu
 Enter price: 600


 All books----------
 Book ID: 101
 Title: abc
 Author: xyz
 Price: 500

 Book ID: 102
 Title: wer
 Author: tyu
 Price: 600
```

```
}
```

```
/*  3.  Vehicle Registration System
       Create a Vehicle class with registration number, owner name, and vehicle type. Use
       constructors to initialize objects. Maintain a static member to count the total number of
       registered vehicles. Store multiple vehicles in an array of objects and display all records.
*/

#include "3.Vehicle_Reg.h"

int main()
{
        int nov;

        cout<<"\n Enter number of vehicles: ";
        cin>>nov;

        Vehicle* varr = new Vehicle[nov];

        for(int i=0; i<nov; i++)
        {
                cout<<"\n Enter details of vehicle "<<i+1;
                varr[i].accept();
        }
        cout<<"\n\n All vehicles----------";

        for(int i=0; i<nov; i++)
        {
                varr[i].display();
                cout<<endl;
        }
        Vehicle::displayCount();
        delete[] varr;
}
```

```
/*
   3.  Vehicle Registration System
       Create a Vehicle class with registration number, owner name, and vehicle type. Use
       constructors to initialize objects. Maintain a static member to count the total number of
       registered vehicles. Store multiple vehicles in an array of objects and display all records.
*/
#pragma once
#include<iostream>
#include<string>
using namespace std;

class Vehicle
{
        private:
                int regno;
                string ownername;
                string vehicletype;

                static int count;

        public:
                Vehicle()
                {
                        regno = 0;
                        ownername = "";
                        vehicletype = "";
                        count++;
                }

                Vehicle(int r, string o, string v)
                {
                        regno = r;
```

```
            ownername = o;
            vehicletype = v;
            count++;
        }
        void accept()
        {
            cout<<"\n Enter registration no: ";
            cin>>regno;

            cout<<" Enter owner name: ";
            cin>>ownername;

            cout<<" Enter vehicle type: ";
            cin>>vehicletype;
        }

        void display()
        {
            cout<<"\n Registration No: "<<regno;
            cout<<"\n Owner Name: "<<ownername;
            cout<<"\n Vehicle Type: "<<vehicletype;
        }

        static void displayCount()
        {
            cout<<"\n Total Registered Vehicles: "<<count;
        }
};
int Vehicle::count = 0;
#pragma once
#include<iostream>
#include<string>
```

```
using namespace std;
class LibraryMember
{
	private:
		int memberid;
		string name;
		int overdue_days;

		static float fine_rate;
	public:
		LibraryMember()
		{
			memberid = 0;
			name = "";
			overdue_days = 0;
		}
		LibraryMember(int id, string n, int days)
		{
			memberid = id;
			name = n;
			overdue_days = days;
		}
		void accept()
		{
			cout<<"\n Enter member id: ";
			cin>>memberid;

			cout<<" Enter name: ";
			cin>>name;

			cout<<" Enter overdue days: ";
			cin>>overdue_days;
```

```
        }

        float calculateFine()
        {
            return overdue_days * fine_rate;
        }
        static void setFineRate()
        {
            cout<<"\n Enter fine rate per day: ";
            cin>>fine_rate;
        }
        static void displayFineRate()
        {
            cout<<"\n Fine rate per day: "<<fine_rate;
        }
        void display()
        {
            cout<<"\n Member ID: "<<memberid;
            cout<<"\n Name: "<<name;
            cout<<"\n Overdue Days: "<<overdue_days;
            cout<<"\n Fine: "<<calculateFine();
        }
};float LibraryMember::fine_rate = 0;
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>g++ 3.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>myexe.exe

 Enter number of vehicles: 2

 Enter details of vehicle 1
 Enter registration no: 101
 Enter owner name: abc
 Enter vehicle type: poi

 Enter details of vehicle 2
 Enter registration no: 102
 Enter owner name: asd
 Enter vehicle type: fgh


 All vehicles------------
 Registration No: 101
 Owner Name: abc
 Vehicle Type: poi

 Registration No: 102
 Owner Name: asd
 Vehicle Type: fgh

 Total Registered Vehicles: 2
```

```
/*  4.  Library Fine Calculator
        Create a LibraryMember class with member ID, name, and number of overdue days. Use a
        constructor to initialize the data. Write a member function to calculate the fine. Use a static
        data member to store the fine rate per day and a static member function to modify or
        display the fine rate.
*/

#include"4.Lib_Fine.h"

int main()
{
        int nom;

        cout<<"\n Enter number of library members: ";
        cin>>nom;

        LibraryMember* marr = new LibraryMember[nom];

        LibraryMember::setFineRate();

        for(int i=0; i<nom; i++)
        {
                cout<<"\n Enter details of member "<<i+1;
                marr[i].accept();
        }

        cout<<"\n\n Fine Rate----------";
        LibraryMember::displayFineRate();

        cout<<"\n\n Member Details----------";

        for(int i=0; i<nom; i++)
```

```
        {
                marr[i].display();
                cout<<endl;
        }

        delete[] marr;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>g++ 4.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\3.CPP>myexe.exe

 Enter number of library members: 2

 Enter fine rate per day: 100

 Enter details of member 1
 Enter member id: 101
 Enter name: asd
 Enter overdue days: 12

 Enter details of member 2
 Enter member id: 102
 Enter name: sdf
 Enter overdue days: 21


 Fine Rate------------
 Fine rate per day: 100

 Member Details------------
 Member ID: 101
 Name: asd
 Overdue Days: 12
 Fine: 1200

 Member ID: 102
 Name: sdf
 Overdue Days: 21
 Fine: 2100
```

# C++ BASIC PROGRAMMING ASSIGNMENT

## DAY 4

```
/*
1. Dynamic Integer Storage

Create a class Number that dynamically allocates memory for an integer using a pointer.
Initialize the value using a constructor, implement a copy constructor to create a deep copy,
and use a destructor to release the dynamically allocated memory.
*/

#include "Number.h"

int main()
{
   Number n1(100);

   cout << "\nNumber n1----------";
   n1.display();

   Number n2(n1);

   cout << "\nNumber n2 - Deep Copy----------";
   n2.display();

   return 0;
}
```

```
#pragma once

#include<iostream>
using namespace std;

class Number
{
private:
   int* no;

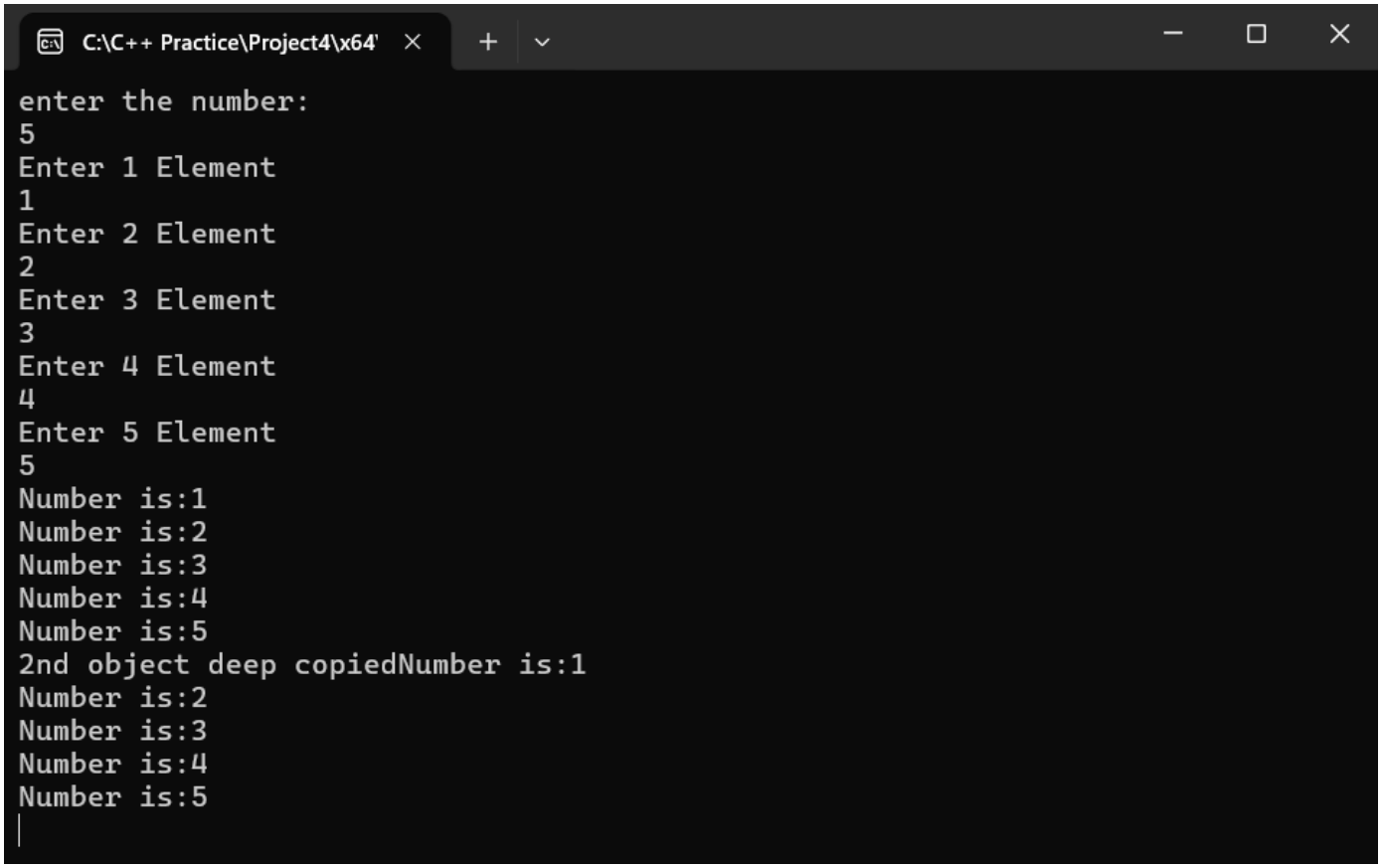
public:

   // Parameterized Constructor
   Number(int no)
   {
      this->no = new int;
      *(this->no) = no;
   }

   // Copy Constructor
   Number(Number& copyObj)
   {
      this->no = new int;
      *(this->no) = *(copyObj.no);
   }

   // Display
   void display()
   {
      cout << "\nThe number is: " << *(this->no);
   }
```

```
    // Destructor
    ~Number()
    {
        if (this->no != NULL)
        {
            delete this->no;
            this->no = NULL;
        }
    }
};
```



/*

2. Library Member Records

Create a LibraryMember class with member ID, name, and number of books issued. Initialize an object using a constructor and create another object using a copy constructor. Use a destructor to display a message when a member object is removed.

*/

```
#include <iostream>
#include "Library.h"

using namespace std;

int main() {
    // 1. Create object using Parameterized Constructor
    LibraryMember member1(101, "Alex", 3);
    member1.accept();
    member1.display();

    // 2. Create another object using Copy Constructor
    LibraryMember member2 = member1;

    cout << "\n--- Displaying Copied Member ---";
    member2.display();

    return 0; // Destructors trigger automatically when objects go out of scope
}
```

```
  #pragma once
  #include <iostream>
  #include <string>

  using namespace std;

  class LibraryMember {
  private:
      int id;
      string name;
      int noOfBooks;
```

```
    int* barr; // Dynamic array to store book IDs

public:
    // Default Constructor
    LibraryMember() {
        id = 0;
        name = "NA";
        noOfBooks = 0;
        barr = nullptr; // Prevents garbage memory access
    }

    // Parameterized Constructor
    LibraryMember(int id, string name, int noOfBooks) {
        this->id = id;
        this->name = name;
        this->noOfBooks = noOfBooks;

        if (this->noOfBooks > 0) {
            barr = new int[this->noOfBooks];
            for (int i = 0; i < this->noOfBooks; i++) {
                barr[i] = 0;
            }
        }
        else {
            barr = nullptr;
        }
    }

    // Copy Constructor (Deep Copy)
    LibraryMember(const LibraryMember& copyObj) {
        this->id = copyObj.id;
        this->name = copyObj.name;
        this->noOfBooks = copyObj.noOfBooks;

        if (this->noOfBooks > 0 && copyObj.barr != nullptr) {
            this->barr = new int[this->noOfBooks];
            for (int i = 0; i < this->noOfBooks; i++) {
                this->barr[i] = copyObj.barr[i];
            }
        }
```

```
        else {
            this->barr = nullptr;
        }
    }

    // Accept book IDs from user
    void accept() {
        if (noOfBooks > 0 && barr != nullptr) {
            cout << "Enter " << noOfBooks << " Book IDs:" << endl;
            for (int i = 0; i < noOfBooks; i++) {
                cin >> barr[i];
            }
        }
    }

    // Display member details and book list
    void display() const {
        cout << "\nMember ID: " << id
             << " | Name: " << name
             << " | Books Issued: " << noOfBooks << endl;

        if (noOfBooks > 0 && barr != nullptr) {
            cout << "Book IDs: ";
            for (int i = 0; i < noOfBooks; i++) {
                cout << barr[i] << " ";
            }
            cout << endl;
        }
    }

    // Destructor
    ~LibraryMember() {
        if (barr != nullptr) {
            delete[] barr;
            barr = nullptr;
        }
        cout << "Member object (ID: " << id << ") removed from memory." << endl;
    }
};
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\4.CPP>g++ 2.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\4.CPP>myexe.exe
Enter 3 Book IDs:
101
202
303

Member ID: 101 | Name: Alex | Books Issued: 3
Book IDs: 101 202 303

--- Displaying Copied Member ---
Member ID: 101 | Name: Alex | Books Issued: 3
Book IDs: 101 202 303
Member object (ID: 101) removed from memory.
Member object (ID: 101) removed from memory.
```

/*

3. Complex Number Addition

Create a Complex class with real and imaginary data members. Initialize the complex numbers using appropriate constructors. Overload the '+' operator to add two Complex objects and display the resulting complex number.

*/

```
#include"complex.h"

int main() {
	complex d1(10.0f, 20.0f);
	d1.display();
	complex d2(10.0f, 20.0f);
	d2.display();

	complex d3(0, 0);
	d3 = d1 + d2;
	d3.display();


}
```

complex.h

```
#pragma once
//Create a Complex class with real and imaginary data members.Initialize the complex
//numbers using appropriate constructors.Overload the '+' operator to add two Complex
//objects and display the resulting complex number.

#include<iostream>
using namespace std;

class complex{
private:
	float real, imag;

public:
	complex(float real, float imag) {
		this->real = real;
		this->imag = imag;
```

```
    }

    void display() {
        cout << real << endl;
        cout << imag << endl;
    }

    complex& operator+(complex& obj) {
        complex temp(0, 0);

        temp.imag = this->imag + obj.imag;
        temp.real = this->real + obj.real;
        return temp;
    }

};
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\4.CPP>g++ 3.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\4.CPP>myexe.exe

First Complex Number:
Real: 10
Imaginary: 20
Second Complex Number:
Real: 10
Imaginary: 20
Addition:
Real: 20
Imaginary: 40
```

```
/*
4. Distance Comparison and Addition

Create a Distance class with feet and inches as data members. Use constructors to initialize
Distance objects. Overload the '+' operator to add two Distance objects and overload the '=='
operator to compare whether two Distance objects are equal. Display the appropriate results.
*/
#include"Distance.h"

int main() {
	Distance d1(10.0f, 20.0f);
	d1.display();

	Distance d2(10.0f, 20.0f);
	d2.display();

	Distance d3(0,0);
	d3 = d1 + d2;
	d3.display();

	if (d1 == d2) {
		cout << "distance is equal"<<endl;
	}
	else {
		cout << "no equal";
	}
}

  #pragma once
  //Create a Complex class with real and imaginary data members.Initialize the complex
  //numbers using appropriate constructors.Overload the '+' operator to add two Complex
  //objects and display the resulting complex number.

  #include<iostream>
  using namespace std;

  class Distance {
  private :
```

```
    float inches, feet;

public:
    Distance(float inches,float feet) {
        this->inches = inches;
        this->feet = feet;
    }
    void display() {
        cout << inches<<endl;
        cout << feet<<endl;
    }
    Distance& operator+(Distance& obj) {
        Distance temp(0, 0);

        temp.feet = this->feet + obj.feet;
        temp.inches = this->inches + obj.inches;
        return temp;
    }
    bool operator==(Distance& obj) {
        if (this->inches == obj.inches && this->feet == obj.feet) {
            return true;
        }
        else {
            return false;
        }
    }
};
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\4.CPP>g++ 4.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\4.CPP>myexe.exe

Distance d1------------
Feet: 10
Inches: 20
Distance d2------------
Feet: 10
Inches: 20
Addition d3------------
Feet: 23.3333
Inches: 4
Distance is equal
```

# C++ BASIC PROGRAMMING ASSIGNMENT

## DAY 5

```
/*
Question 1: Containment of Objects – Online Shopping Order Management System

Problem Statement:
Develop a C++ program for an Online Shopping Order Management System
to demonstrate containment of objects. Create Customer and Product classes,
and an Order class containing objects of both classes. Initialize the contained
objects through constructors, calculate the total order amount, and display
complete order details.
*/

#include "Order.h"

int main()
{
   Order o1(1, "abc", 9763770278, 213, "Apple", 100000, 2, 101);
   o1.displayOrder();

   Order o2(2, "xyz", 9876543210, 214, "Lenovo", 50000, 3, 102);
   o2.displayOrder();

   Order o3(3, "pqr", 9123456789, 215, "Samsung", 70000, 1, 103);
   o3.displayOrder();

   return 0;
   }
   #pragma once

   #include<iostream>
   #include<string>
   using namespace std;

   class Customer
   {
   private:
      int customerNo;
```

```
        string name;
        long long phoneNo;

    public:

        Customer()
        {
            customerNo = 0;
            name = "NA";
            phoneNo = 0;
        }

        Customer(int cno, string name, long long phone)
        {
            customerNo = cno;
            this->name = name;
            phoneNo = phone;
        }

        void displayCustomer()
        {
            cout << "\nCustomer Number: " << customerNo;
            cout << "\nName: " << name;
            cout << "\nPhone Number: " << phoneNo;
        }
    };
#pragma once

#include<iostream>
#include<string>
using namespace std;

class Product
{
private:
    int productId;
    string productName;
    float price;
    int quantity;
```

```
public:

    Product()
    {
        productId = 0;
        productName = "NA";
        price = 0.0f;
        quantity = 0;
    }

    Product(int productId, string productName, float price, int quantity)
    {
        this->productId = productId;
        this->productName = productName;
        this->price = price;
        this->quantity = quantity;
    }

    void displayProduct()
    {
        cout << "\nProduct Id: " << productId;
        cout << "\nProduct Name: " << productName;
        cout << "\nProduct Price: " << price;
        cout << "\nQuantity: " << quantity;
    }

    float getTotal()
    {
        return price * quantity;
    }
};

#pragma once

#include<iostream>
#include<string>
#include"Customer.h"
#include"Product.h"
using namespace std;
```

```
class Order
{
private:
   int orderId;
   Customer c1;
   Product p1;

public:

   Order()
   {
      orderId = 0;
   }

   Order(int cno, string name, long long phone,
         int proid, string pname, float pprice,
         int quantity, int ord)
      : c1(cno, name, phone),
        p1(proid, pname, pprice, quantity)
   {
      orderId = ord;
   }

   void displayOrder()
   {
      cout << "\nOrder Id: " << orderId;

      c1.displayCustomer();

      p1.displayProduct();

      cout << "\nTotal Order Amount: " << p1.getTotal();

      cout << "\n";
   }
};
```

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

D:\CDAC\ASSIGNMENTS\College_Assignments\5.CPP>g++ 1.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\5.CPP>myexe.exe

Order Id: 101
Customer Number: 1
Name: abc
Phone Number: 9763770278
Product ID: 213
Product Name: Apple
Product Price: 100000
Quantity: 2
Total Order Amount: 200000

Order Id: 102
Customer Number: 2
Name: xyz
Phone Number: 9876543210
Product ID: 214
Product Name: Lenovo
Product Price: 50000
Quantity: 3
Total Order Amount: 150000

Order Id: 103
Customer Number: 3
Name: pqr
Phone Number: 9123456789
Product ID: 215
Product Name: Samsung
Product Price: 70000
Quantity: 1
Total Order Amount: 70000
```

# C++ BASIC PROGRAMMING ASSIGNMENT

## DAY 6

/*

1. Problem 1: Hospital Billing System Using Inheritance, Polymorphism and RTTI

A hospital provides treatment to different categories of patients. The billing process differs depending on the type of patient and the services provided.

Develop a Hospital Billing System using object-oriented programming concepts in C++. The system should maintain patient information and calculate the final hospital bill according to the patient's category.

Create a base class named Patient that stores common details such as Patient ID, Patient Name, Age and Number of Days Admitted.

Derive InPatient and OutPatient from Patient. Further derive EmergencyPatient from InPatient. An InPatient may be charged for room charges, doctor consultation, medicines and treatment. An OutPatient may be charged for consultation, diagnostic tests and medicines. An EmergencyPatient should include inpatient charges along with emergency service charges and ambulance charges, if applicable.

The program should calculate and display the bill differently for each type of patient.

Requirements:
• Create a base class Patient with common data members such as patientId, patientName, age and numberOfDaysAdmitted.
• Provide appropriate constructors to initialize patient information.
• Create virtual functions such as calculateBill() and displayPatientDetails().
• Derive InPatient and OutPatient from Patient, and derive EmergencyPatient from InPatient.
• Override calculateBill() in InPatient, OutPatient and EmergencyPatient.
• Use suitable billing formulas for each patient category.
• Maintain different patient objects using Patient pointers, such as an array or vector of Patient pointers.
• Demonstrate runtime polymorphism by calling calculateBill() and displayPatientDetails() through Patient pointers.
• Use RTTI with dynamic_cast or typeid to identify the actual runtime type of each patient object.
• If an object is an EmergencyPatient, display additional information such as emergency charges, ambulance charges and type of emergency.

• Perform safe casting by checking the result of dynamic_cast before accessing derived-class-specific members.
• Include a virtual destructor in the Patient class.
• Create a menu-driven application with options to add different patient types, display all patients, calculate bills and identify patient types using RTTI.
• Ensure that all dynamically created objects are properly deleted before program termination.

*/

```
#pragma once

#include<iostream>
using namespace std;

class Patient {
protected:
    static int count;
    int id;
    string name;
    int age;
    double consultationFees;

public:
    Patient(string name, int age, double consultationFees) {
        count++;
        this->id = count;
        this->name = name;
        this->age = age;
        this->consultationFees = consultationFees;
    }

    virtual void displayDetails() {
        cout << "The deatils-" << endl;
        cout << "Patient id-" << id << endl;
        cout << "Patient name-" << name << endl;
        cout << "Patient age-" << age << endl;
        cout << "Consultation fee-" << consultationFees << endl;

    }

    virtual double calculateBill() = 0;
```

```
};

int Patient::count = 0;


#pragma once
#include"Patient.h"

class InPatient :public Patient {
private:
	double roomCharges;
	int numberOfDayAdmitted;

public:
	InPatient(string name, int age, double consultationFees, double roomCharges, int
numberOfDayAdmitted)
		:Patient(name, age, consultationFees) {
		this->roomCharges = roomCharges;
		this->numberOfDayAdmitted = numberOfDayAdmitted;
	}

	void displayDetails() {
		Patient::displayDetails();
		cout << "Room chareges are: " << roomCharges << endl;
		cout << "Number of Days admitted: " << numberOfDayAdmitted << endl;
	}

	double calculateBill()
	{
		return (roomCharges * numberOfDayAdmitted) + consultationFees;
	}

	void addCharges() {
		cout << "Additional charges applied";

	}
};
#pragma once
#include"Patient.h"
```

```
class OutPatient :public Patient {
private:
	double medicineCharges;

public:
	OutPatient(string name, int age, double consultationFees, double medicineCharges)
		:Patient(name, age, consultationFees) {
		this->medicineCharges = medicineCharges;
	}

	void displayDetails() {
		Patient::displayDetails();
		cout << "Medicine charges: " << medicineCharges;
	}

	double calculateBill()
	{
		return medicineCharges + consultationFees;
	}
	void addCharges() {
		cout << "Additional charges applied";

	}
};



#pragma once
#include"InPatient.h"

class EmergencyPatient :public InPatient {
private:
	double ambulancecharges;
	double emergencycharges;

public:
	EmergencyPatient(string name, int age, double consultationFees, double roomCharges,
int numberOfDayAdmitted, double ambulancecharges, double emergencycharges)
		:InPatient(name, age, consultationFees, roomCharges, numberOfDayAdmitted) {
		this->ambulancecharges = ambulancecharges;
```

```
		this->emergencycharges = emergencycharges;
	}

	void displayDetails() {
		InPatient::displayDetails();
		cout << "Ambulance chareges are: " << ambulancecharges << endl;
		cout << "Emergency charges: " << emergencycharges << endl;
	}

	double calculateBill()
	{
		return InPatient::calculateBill() + ambulancecharges + emergencycharges;
	}

	void notify() {
		cout << "Emergency charges are applied";
	}

	void addCharges() {
		cout << "Additional charges applied";

	}
};


#include"InPatient.h"
#include"OutPatient.h"
#include"EmergencyPatient.h"


class PrintoScreen
{
public:
	static void displayDetails(Patient* ptr)
	{
		ptr->displayDetails();

		//RTTI- run time type identification and downcasting
		//downcasting is performed for invoking special methods using derived class
pointer
```

```
		//typeid operator check the runtime type of eptr

		if (typeid(*ptr) == typeid(InPatient))
		{
			InPatient* iptr = dynamic_cast<InPatient*>(ptr);
			//dynamic cast operator perform a downcasting
			//of a generic pointer eptr type Employee* to Manager*
			iptr->addCharges();
		}

		if (typeid(*ptr) == typeid(OutPatient))
		{
			OutPatient* mptr = dynamic_cast<OutPatient*>(ptr);
			//dynamic cast operator perform a downcasting
			//of a generic pointer eptr type Employee* to Manager*
			mptr->addCharges();
		}

		if (typeid(*ptr) == typeid(EmergencyPatient))
		{
			EmergencyPatient* eptr = dynamic_cast<EmergencyPatient*>(ptr);
			//dynamic cast operator perform a downcasting
			//of a generic pointer eptr type Employee* to Manager*
			eptr->addCharges();
		}
	}
};

int main()
{
	Patient** ptr;

	string name;
	int age;
	double consultationFees;
	double ambulancecharges;
	double emergencycharges;
	double medicineCharges;
	double roomCharges;
```

```
int numberOfDayAdmitted;

int nop;

cout << "Enter the number of patients : ";
cin >> nop;

// Dynamically create an array of Patient pointers
ptr = new Patient * [nop];

for (int i = 0; i < nop; i++)
{
        int choice;

        cout << "\n1. InPatient";
        cout << "\n2. OutPatient";
        cout << "\n3. EmergencyPatient";
        cout << "\nEnter your choice : ";
        cin >> choice;

        cout << "Enter patient name : ";
        cin >> name;

        cout << "Enter age : ";
        cin >> age;

        cout << "Enter consultation fees : ";
        cin >> consultationFees;

        switch (choice)
        {
        case 1:
        {
                cout << "Enter room charges : ";
                cin >> roomCharges;

                cout << "Enter number of days admitted : ";
                cin >> numberOfDayAdmitted;

                // Parent pointer pointing to InPatient object
```

```
            ptr[i] = new InPatient(
                        name,
                        age,
                        consultationFees,
                        roomCharges,
                        numberOfDayAdmitted
            );

            break;
}

case 2:
{
            cout << "Enter medicine charges : ";
            cin >> medicineCharges;

            // Parent pointer pointing to OutPatient object
            ptr[i] = new OutPatient(
                        name,
                        age,
                        consultationFees,
                        medicineCharges
            );

            break;
}

case 3:
{
            cout << "Enter room charges : ";
            cin >> roomCharges;

            cout << "Enter number of days admitted : ";
            cin >> numberOfDayAdmitted;

            cout << "Enter ambulance charges : ";
            cin >> ambulancecharges;

            cout << "Enter emergency charges : ";
            cin >> emergencycharges;
```

```
                // Parent pointer pointing to EmergencyPatient object
                ptr[i] = new EmergencyPatient(
                        name,
                        age,
                        consultationFees,
                        roomCharges,
                        numberOfDayAdmitted,
                        ambulancecharges,
                        emergencycharges
                );

                break;
        }

        default:
                cout << "Invalid choice!";
                i--;    // repeat this patient's input
        }
}

// Display details and calculate bill
for (int i = 0; i < nop; i++)
{
        cout << "\n\n-----------------------------";

        PrintoScreen::displayDetails(ptr[i]);

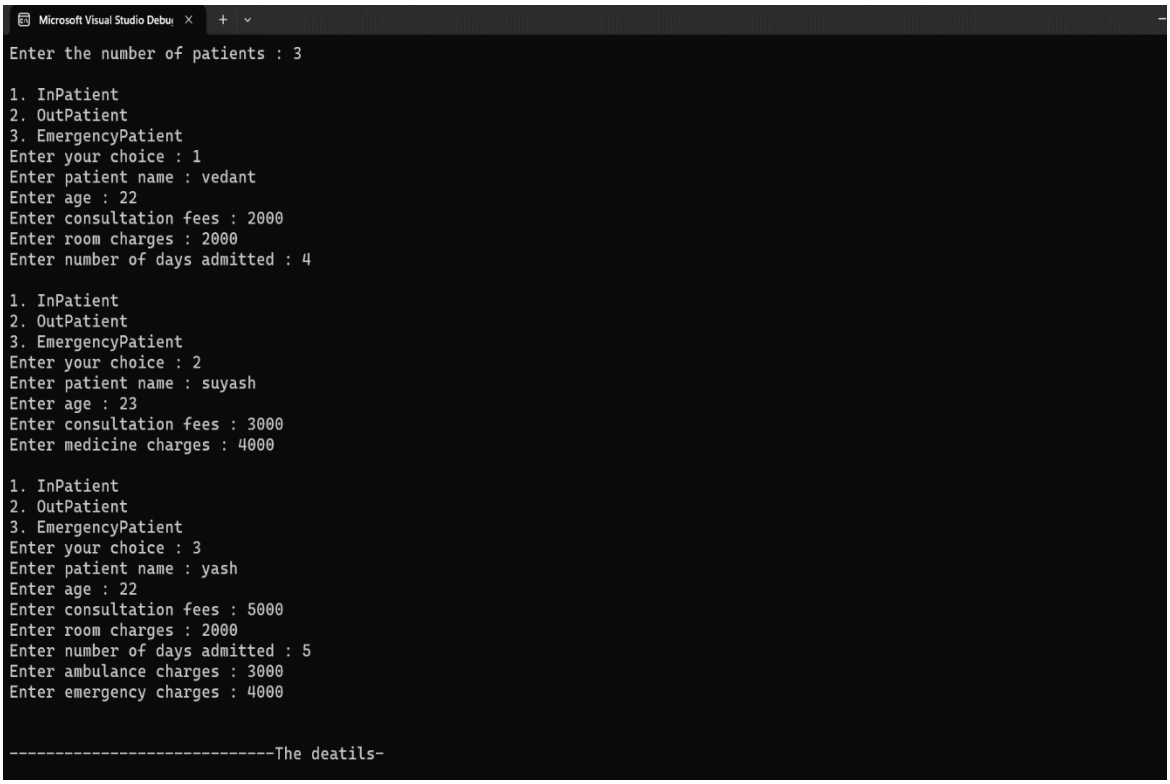
        cout << "\n Calculated Bill = "
                << ptr[i]->calculateBill();
}

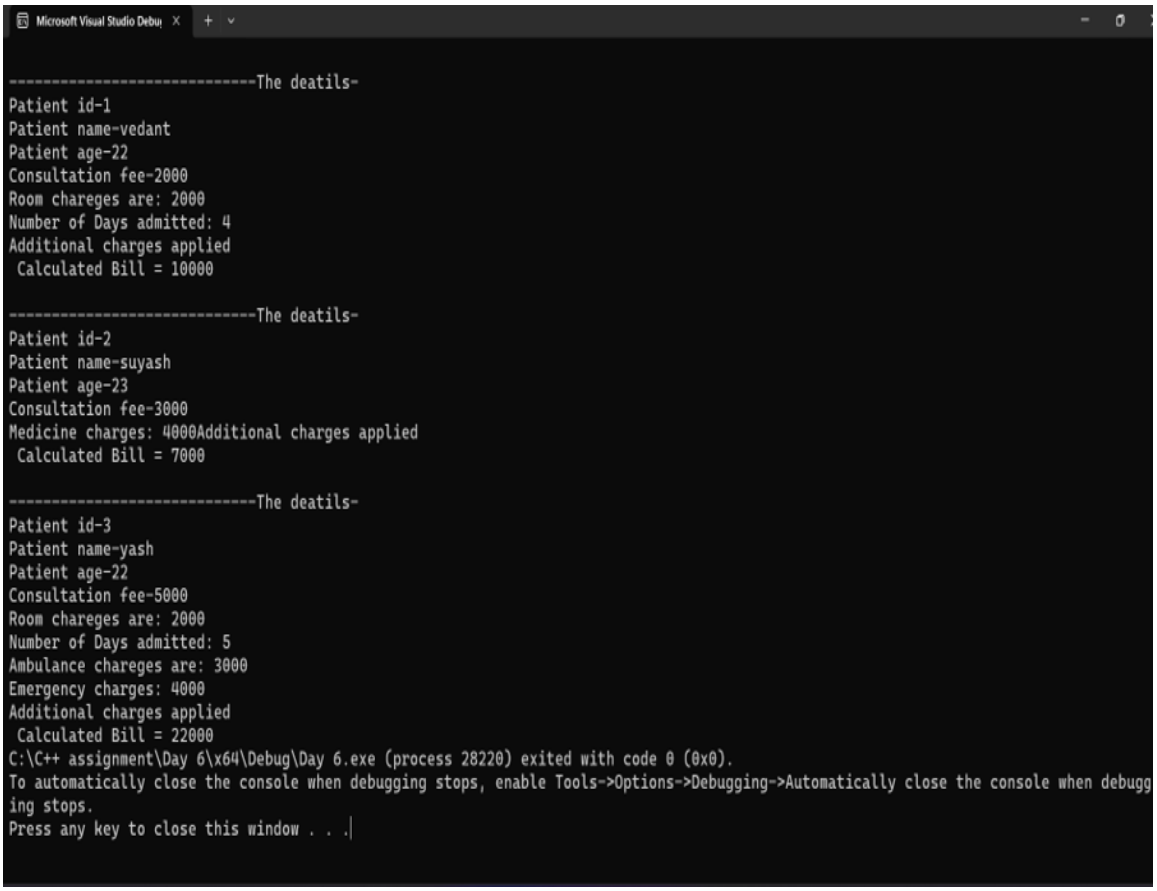


// Delete every dynamically created object
for (int i = 0; i < nop; i++)
{
        delete ptr[i];
}
```

```
    // Delete the array of pointers
    delete[] ptr;

    return 0;
}
```




Microsoft Visual Studio Debu
----------The deatils-
Patient id-1
Patient name-vedant
Patient age-22
Consultation fee-2000
Room chareges are: 2000
Number of Days admitted: 4
Additional charges applied
Calculated Bill = 10000
----------The deatils-
Patient id-2
Patient name-suyash
Patient age-23
Consultation fee-3000
Medicine charges: 4000Additional charges applied
Calculated Bill = 7000
----------The deatils-
Patient id-3
Patient name-yash
Patient age-22
Consultation fee-5000
Room chareges are: 2000
Number of Days admitted: 5
Ambulance chareges are: 3000
Emergency charges: 4000
Additional charges applied
Calculated Bill = 22000
C:\C++ assignment\Day 6\x64\Debug\Day 6.exe (process 28220) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugg
ing stops.
Press any key to close this window . . .

/*

2. Problem 2: Financial Account Management System Using Inheritance, Polymorphism and RTTI

A bank provides different types of accounts to its customers. Each account contains common information, but the rules for calculating returns, handling withdrawals, overdraft facilities and maturity amounts differ depending on the account type.

Develop a Financial Account Management System using inheritance, runtime polymorphism and RTTI.

Create a base class named Account containing common information such as Account Number, Customer Name and Balance.

Derive SavingsAccount, CurrentAccount and FixedDepositAccount from Account. A savings account earns interest based on its balance. A current account may provide an overdraft facility and apply service charges. A fixed deposit account earns returns based on principal amount, interest rate and deposit tenure.

The program should process all account types through a common base-class interface while also identifying the actual account type at runtime.

Requirements:
• Create a base class Account with data members such as accountNumber, customerName and balance.
• Provide suitable constructors for initializing account information.
• Declare virtual functions calculateReturns() and displayAccountInfo().
• Derive SavingsAccount, CurrentAccount and FixedDepositAccount from Account.
• Include additional data in SavingsAccount such as interestRate and minimumBalance.
• Override calculateReturns() to calculate interest earned by a savings account.
• Include overdraftLimit and serviceCharges in CurrentAccount and implement suitable withdrawal rules.
• Include principal amount, interest rate and deposit tenure in FixedDepositAccount and calculate the maturity amount.
• Store objects of all account types using Account pointers.
• Demonstrate runtime polymorphism by calling calculateReturns() and displayAccountInfo() through Account pointers.
• Use dynamic_cast to identify the actual runtime account type.
• For SavingsAccount, display interest rate and minimum balance.

• For CurrentAccount, display overdraft limit and service charges.
• For FixedDepositAccount, display interest rate, deposit tenure and calculated maturity amount.
• Perform safe downcasting by checking the result of dynamic_cast before accessing derived-class-specific members.
• Create a menu-driven application to create accounts, display accounts, calculate returns, perform withdrawals and identify account types using RTTI.

*/

```
#pragma once

#include<iostream>
#include<string>
using namespace std;

class Account {
protected:
	int AccountNumber;
	string customerName;
	int Balance;

public:
	Account()
	{
		AccountNumber = 0;
		customerName = "NA";
		Balance = 0;
	}
	Account(int AccountNumber,string customerName,int Balance){
	this->AccountNumber = AccountNumber;
	this->customerName = customerName;
	this->Balance = Balance;
	}

	virtual void calculateReturns() = 0;

	virtual void displayAccountInfo() {
		cout << "Account number" << AccountNumber;
		cout << "Customer Name" << customerName;
		cout << "Balance" << Balance;
}
```

```
	virtual void withdrawal(double amount) {
		if (amount <= Balance)
		{
			Balance = Balance - amount;
			cout << "\nWithdrawal successful";
		}
		else
		{
			cout << "\nInsufficient balance";
		}
	}
	virtual ~Account()
	{
	}
};


#pragma once
#include"Account.h"

class SavingAccount:public Account {

private:
	double interestRate;
	double minimumBalance;

public:
	SavingAccount() {
		interestRate = 0;
		minimumBalance = 0;
	}

	SavingAccount(int AccountNumber, string customerName, int Balance, double
interestRate,double minimumBalance)
		:Account(AccountNumber, customerName, Balance) {
		this->interestRate = interestRate;
		this->minimumBalance = minimumBalance;
	}
```

```
        void calculateReturns() override {
                double interest= Balance * interestRate / 100;

                cout << "\nInterest Earned : " << interest;
        }

        void displayAccountInfo() override {
                Account::displayAccountInfo();
                cout << "Interest rate:" << interestRate;
                cout << "Minimum balance:" << minimumBalance;
        }

        void withdrawal(double amount) override {
                if (Balance - amount <= minimumBalance) {
                        Balance = Balance - amount;
                        cout << "\nSavings withdrawal successful";
                }
                else
                {
                        cout << "\nCannot withdraw. Minimum balance must be maintained.";
                }
        }
};


#pragma once
#include"Account.h"

class CurrentAccount : public Account {

private:
        double overdraftLimit;
        double serviceCharges;

public:
        CurrentAccount() {
                overdraftLimit=0;
           serviceCharges=0;
        }
```

```
    CurrentAccount(int AccountNumber, string customerName, int Balance, double
overdraftLimit, double serviceCharges)
        :Account(AccountNumber, customerName, Balance) {
        this->overdraftLimit = overdraftLimit;
        this->serviceCharges = serviceCharges;
    }

    void calculateReturns() override {
        cout << "\nCurrent Account has no interest.";
        cout << "\nService Charges : " << serviceCharges;
    }

    void displayAccountInfo() override {
        Account::displayAccountInfo();
        cout << " overdraft Limit:" << overdraftLimit;
        cout << " service Charges:" << serviceCharges;
    }


    void withdrawal(double amount) override {
        if (amount <= Balance+overdraftLimit) {
            Balance = Balance - amount;

            // Service charge is applied after withdrawal
            Balance = Balance - serviceCharges;
            cout << "\nCurrent account withdrawal successful";
        }
        else
        {
            cout << "\nWithdrawal exceeds overdraft limit.";
        }
    }

};
```

```
#pragma once
#include"Account.h"

class FixedDepositAccount : public Account
{
private:
	double principalAmount;
	double interestRate;
	int depositTenure;

public:

	FixedDepositAccount()
	{
		principalAmount = 0;
		interestRate = 0;
		depositTenure = 0;
	}

	FixedDepositAccount(int AccountNumber, string customerName,int Balance,
		double principalAmount, double interestRate, int depositTenure)
		: Account(AccountNumber, customerName, Balance)
	{
		this->principalAmount = principalAmount;
		this->interestRate = interestRate;
		this->depositTenure = depositTenure;
	}

	void calculateReturns() override
	{
		// Simple interest calculation
		double interest = principalAmount *
			interestRate * depositTenure / 100;

		double maturityAmount = principalAmount + interest;

		cout << "\nInterest Earned  : " << interest;
		cout << "\nMaturity Amount  : " << maturityAmount;
	}
```

```
        void displayAccountInfo() override
        {
                Account::displayAccountInfo();

                cout << "\nPrincipal Amount : " << principalAmount;
                cout << "\nInterest Rate    : " << interestRate;
                cout << "\nDeposit Tenure   : " << depositTenure << " years";

                double interest = principalAmount *
                        interestRate * depositTenure / 100;

                double maturityAmount = principalAmount + interest;

                cout << "\nMaturity Amount  : " << maturityAmount;
        }

        void withdrawal(double amount) override
        {
                cout << "\nWithdrawal is not allowed from Fixed Deposit.";
        }
};
```

```
#include<iostream>
#include"Account.h"
#include"SavingAccount.h"
#include"CurrentAccount.h"
#include"FixedDepositAccount.h"

using namespace std;

int main()
{
   // Base class pointer array
   Account* accounts[10];

   int count = 0;
   int choice;
```

```
do
{
   cout << "\n\n========== BANK ACCOUNT MANAGEMENT ==========";
   cout << "\n1. Create Saving Account";
   cout << "\n2. Create Current Account";
   cout << "\n3. Create Fixed Deposit Account";
   cout << "\n4. Display All Accounts";
   cout << "\n5. Calculate Returns";
   cout << "\n6. Withdraw";
   cout << "\n7. Identify Account Type using RTTI";
   cout << "\n8. Exit";

   cout << "\nEnter your choice : ";
   cin >> choice;

   switch (choice)
   {
   case 1:
   {
      int AccountNumber;
      string customerName;
      int Balance;
      double interestRate;
      double minimumBalance;

      cout << "\nEnter Account Number : ";
      cin >> AccountNumber;

      cout << "Enter Customer Name : ";
      cin >> customerName;

      cout << "Enter Balance : ";
      cin >> Balance;

      cout << "Enter Interest Rate : ";
      cin >> interestRate;

      cout << "Enter Minimum Balance : ";
      cin >> minimumBalance;
```

```
    accounts[count] = new SavingAccount(
        AccountNumber,
        customerName,
        Balance,
        interestRate,
        minimumBalance
    );

    count++;

    cout << "\nSaving Account Created Successfully";

    break;
}

case 2:
{
    int AccountNumber;
    string customerName;
    int Balance;
    double overdraftLimit;
    double serviceCharges;

    cout << "\nEnter Account Number : ";
    cin >> AccountNumber;

    cout << "Enter Customer Name : ";
    cin >> customerName;

    cout << "Enter Balance : ";
    cin >> Balance;

    cout << "Enter Overdraft Limit : ";
    cin >> overdraftLimit;

    cout << "Enter Service Charges : ";
    cin >> serviceCharges;

    accounts[count] = new CurrentAccount(
        AccountNumber,
```

```
            customerName,
            Balance,
            overdraftLimit,
            serviceCharges
        );

        count++;

        cout << "\nCurrent Account Created Successfully";

        break;
    }

    case 3:
    {
        int AccountNumber;
        string customerName;
        int Balance;
        double principalAmount;
        double interestRate;
        int depositTenure;

        cout << "\nEnter Account Number : ";
        cin >> AccountNumber;

        cout << "Enter Customer Name : ";
        cin >> customerName;

        cout << "Enter Balance : ";
        cin >> Balance;

        cout << "Enter Principal Amount : ";
        cin >> principalAmount;

        cout << "Enter Interest Rate : ";
        cin >> interestRate;

        cout << "Enter Deposit Tenure : ";
        cin >> depositTenure;
```

```
        accounts[count] = new FixedDepositAccount(
            AccountNumber,
            customerName,
            Balance,
            principalAmount,
            interestRate,
            depositTenure
        );

        count++;

        cout << "\nFixed Deposit Account Created Successfully";

        break;
    }

    case 4:
    {
        cout << "\n\n========== ALL ACCOUNTS ==========";

        for (int i = 0; i < count; i++)
        {
            cout << "\n\nAccount " << i + 1;

            // Runtime Polymorphism
            accounts[i]->displayAccountInfo();
        }

        break;
    }

    case 5:
    {
        cout << "\n\n========== ACCOUNT RETURNS ==========";

        for (int i = 0; i < count; i++)
        {
            cout << "\n\nAccount " << i + 1;

            // Runtime Polymorphism
```

```
            accounts[i]->calculateReturns();
        }

        break;
    }

    case 6:
    {
        int accountIndex;
        double amount;

        cout << "\nEnter Account Number from list (1-"
            << count << ") : ";
        cin >> accountIndex;

        if (accountIndex >= 1 && accountIndex <= count)
        {
            cout << "Enter withdrawal amount : ";
            cin >> amount;

            // Runtime polymorphism
            accounts[accountIndex - 1]->withdrawal(amount);
        }
        else
        {
            cout << "\nInvalid Account Index";
        }

        break;
    }

    case 7:
    {
        int accountIndex;

        cout << "\nEnter Account Index (1-"
            << count << ") : ";
        cin >> accountIndex;

        if (accountIndex >= 1 && accountIndex <= count)
```

```
    {
        Account* ptr = accounts[accountIndex - 1];

        // RTTI - identify SavingAccount
        SavingAccount* sptr =
            dynamic_cast<SavingAccount*>(ptr);

        if (sptr != nullptr)
        {
            cout << "\nAccount Type : Saving Account";
        }

        // RTTI - identify CurrentAccount
        CurrentAccount* cptr =
            dynamic_cast<CurrentAccount*>(ptr);

        if (cptr != nullptr)
        {
            cout << "\nAccount Type : Current Account";
        }

        // RTTI - identify FixedDepositAccount
        FixedDepositAccount* fptr =
            dynamic_cast<FixedDepositAccount*>(ptr);

        if (fptr != nullptr)
        {
            cout << "\nAccount Type : Fixed Deposit Account";
        }
    }
    else
    {
        cout << "\nInvalid Account Index";
    }

    break;
}

case 8:
{
```

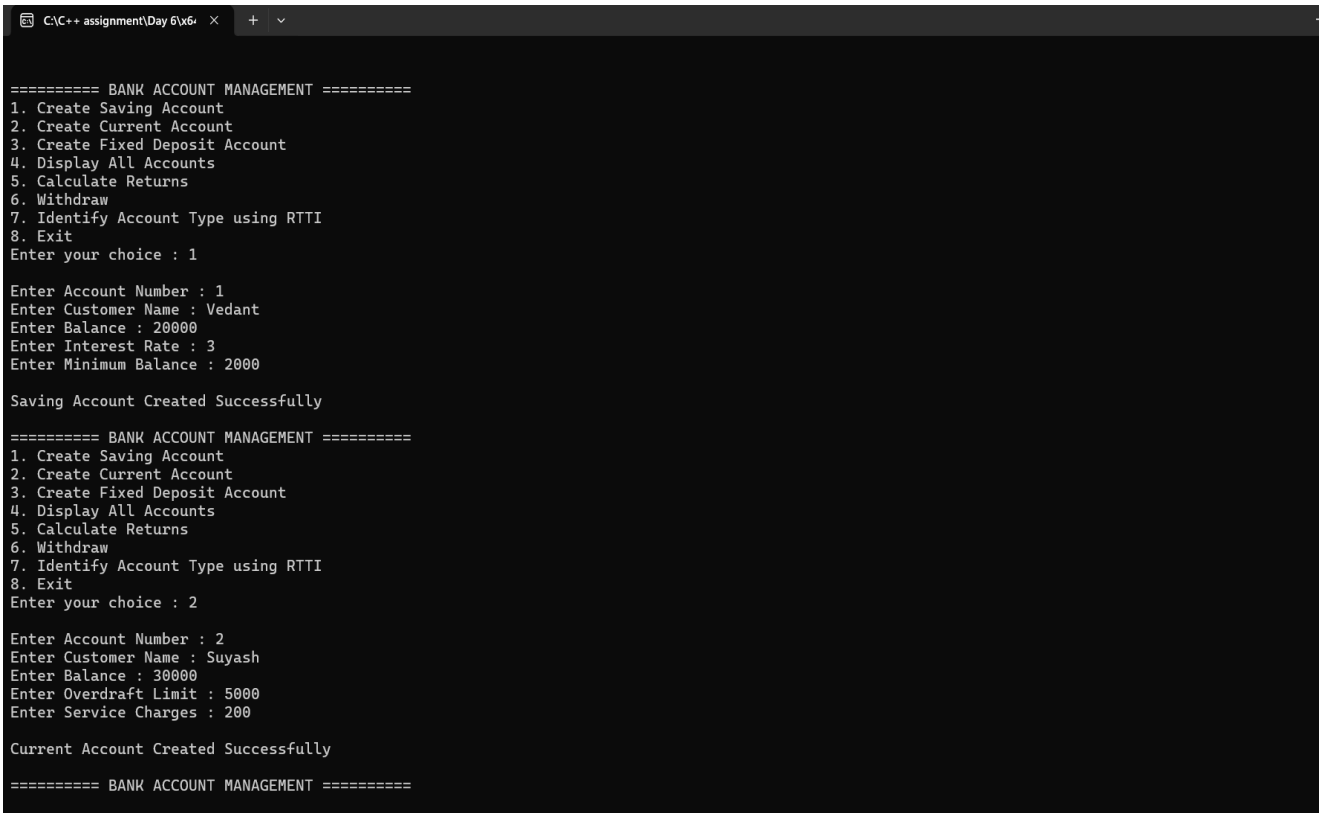
```
            cout << "\nThank you!";
            break;
        }

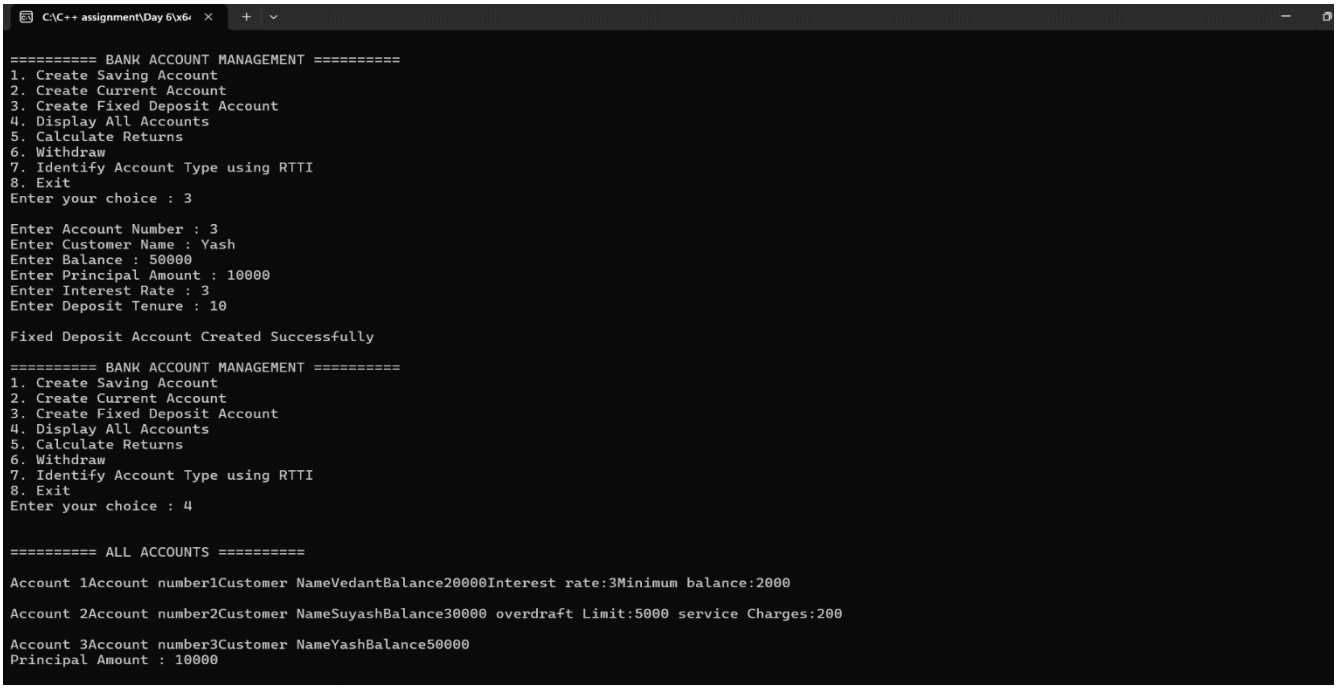
        default:
        {
            cout << "\nInvalid choice";
        }
        }

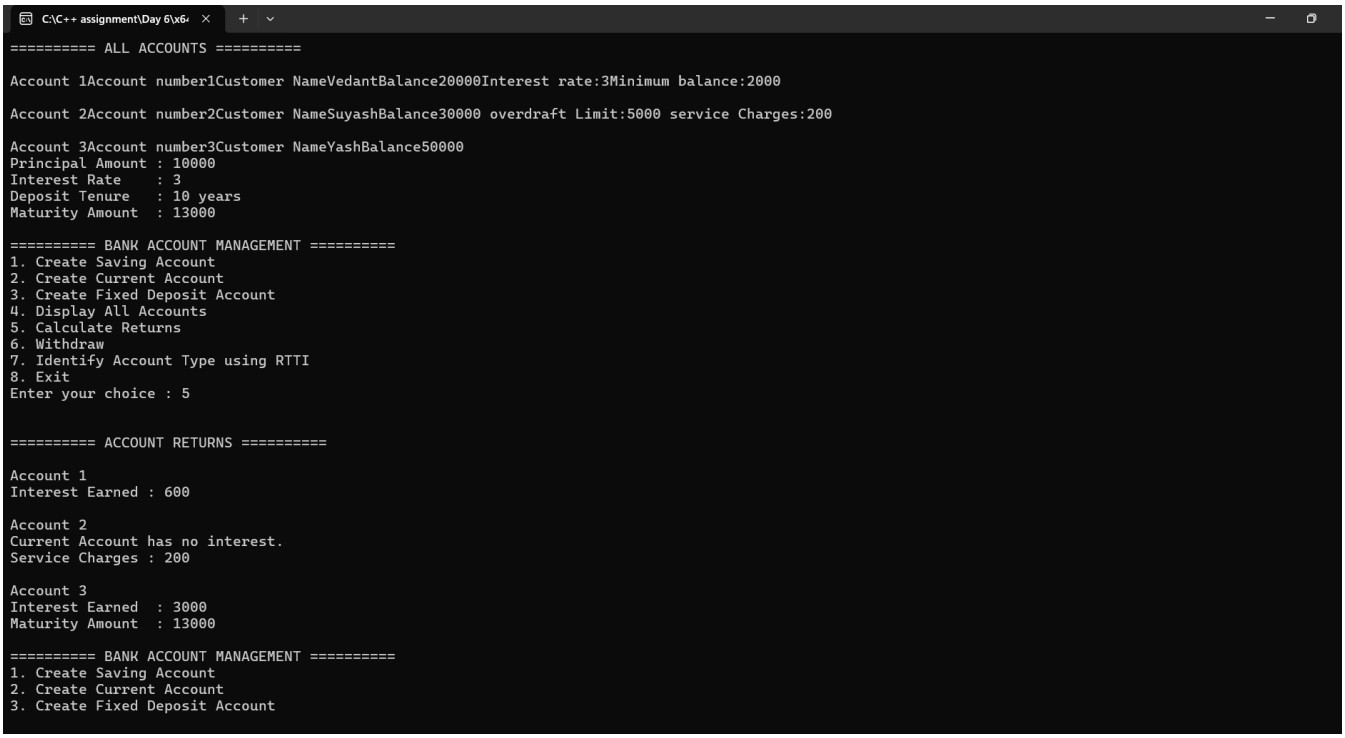
    } while (choice != 8);

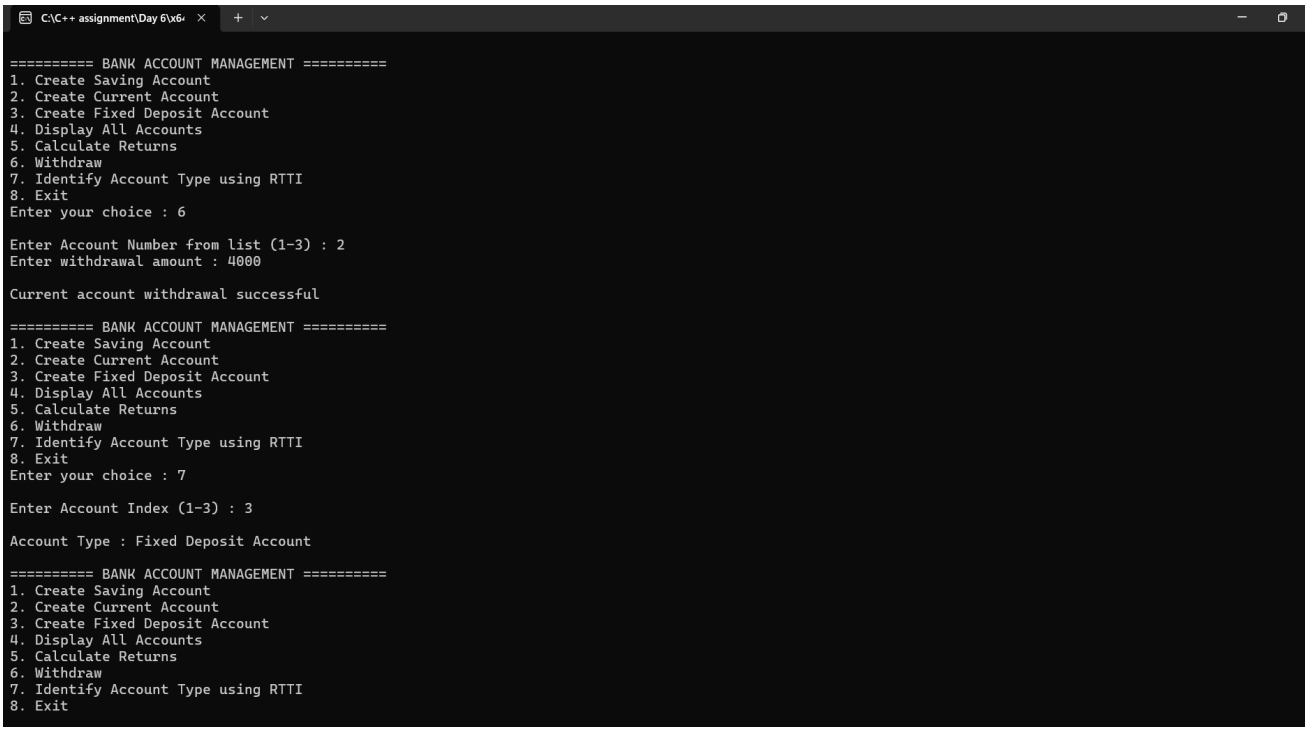

    // Delete dynamically created objects
    for (int i = 0; i < count; i++)
    {
        delete accounts[i];
    }

    return 0;
}
```


C:\C++ assignment\Day 6\x64
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit
Enter your choice : 1
Enter Account Number : 1
Enter Customer Name : Vedant
Enter Balance : 20000
Enter Interest Rate : 3
Enter Minimum Balance : 2000
Saving Account Created Successfully
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit
Enter your choice : 2
Enter Account Number : 2
Enter Customer Name : Suyash
Enter Balance : 30000
Enter Overdraft Limit : 5000
Enter Service Charges : 200
Current Account Created Successfully
========== BANK ACCOUNT MANAGEMENT ==========


C:\C++ assignment\Day 6\x64
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit
Enter your choice : 3
Enter Account Number : 3
Enter Customer Name : Yash
Enter Balance : 50000
Enter Principal Amount : 10000
Enter Interest Rate : 3
Enter Deposit Tenure : 10
Fixed Deposit Account Created Successfully
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit
Enter your choice : 4
========== ALL ACCOUNTS ==========
Account 1Account number1Customer NameVedantBalance20000Interest rate:3Minimum balance:2000
Account 2Account number2Customer NameSuyashBalance30000 overdraft Limit:5000 service Charges:200
Account 3Account number3Customer NameYashBalance50000
Principal Amount : 10000


C:\C++ assignment\Day 6\x64
========== ALL ACCOUNTS ==========
Account 1Account number1Customer NameVedantBalance20000Interest rate:3Minimum balance:2000
Account 2Account number2Customer NameSuyashBalance30000 overdraft Limit:5000 service Charges:200
Account 3Account number3Customer NameYashBalance50000
Principal Amount : 10000
Interest Rate : 3
Deposit Tenure : 10 years
Maturity Amount : 13000
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit
Enter your choice : 5
========== ACCOUNT RETURNS ==========
Account 1
Interest Earned : 600
Account 2
Current Account has no interest.
Service Charges : 200
Account 3
Interest Earned : 3000
Maturity Amount : 13000
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account


C:\C++ assignment\Day 6\x64
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit
Enter your choice : 6
Enter Account Number from list (1-3) : 2
Enter withdrawal amount : 4000
Current account withdrawal successful
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit
Enter your choice : 7
Enter Account Index (1-3) : 3
Account Type : Fixed Deposit Account
========== BANK ACCOUNT MANAGEMENT ==========
1. Create Saving Account
2. Create Current Account
3. Create Fixed Deposit Account
4. Display All Accounts
5. Calculate Returns
6. Withdraw
7. Identify Account Type using RTTI
8. Exit

C++ BASIC PROGRAMMING ASSIGNMENT

DAY 7

```
/*

1. Airline Baggage & Boarding Pass Fee Engine — Abstract Classes & Polymorphism

*/

#pragma once
#include<iostream>
#include<string>
using namespace std;
class FlightTicket {
protected:
	string passengerName;
	string flightNumber;
	double baseFare;
	int baggageCount;
	double totalWeightKg;

public:
	FlightTicket(string passengerName,
		string flightNumber,
		double baseFare,
		int baggageCount,
		double totalWeightKg)
	{
		this->passengerName = passengerName;
		this->flightNumber = flightNumber;
		this->baseFare = baseFare;
		this->baggageCount = baggageCount;
		this->totalWeightKg = totalWeightKg;
	}

	int getBaggageCount() const { return baggageCount; }
	double getTotalWeightKg() const { return totalWeightKg; }
```

```
        virtual double calculateTotalFare() = 0;

        virtual void printBoardingPass() = 0;
};
```

Code- EconomyTicket.h

```
#pragma once
#include"FlightTicket.h"

class EconomyTicket :public FlightTicket
{
private:
        double extraBagFeePerKg;
        double seatSelectionFee;

        int baggageCount;
        int totalWeightKg;

        double baggageCharge;
        double totalFare;

public:

        EconomyTicket(string passengerName,
                string flightNumber,
                double baseFare, int baggageCount, double totalWeightKg, double
extraBagFeePerKg, double seatSelectionFee)
                :FlightTicket(passengerName, flightNumber, baseFare, baggageCount,
totalWeightKg)
        {
                this->baggageCount = baggageCount;
                this->totalWeightKg = totalWeightKg;
                this->extraBagFeePerKg = extraBagFeePerKg;
                this->seatSelectionFee = seatSelectionFee;
                this-> baggageCharge=0;
                this->totalFare=0;
        }

        double calculateTotalFare() override {
                baggageCharge = 0;
```

```
        int bags = getBaggageCount();
        double weight = getTotalWeightKg();

        if (bags > 1) {
                double extraBagFeeCount = bags - 1;
                baggageCharge = extraBagFeeCount * 50;
        }

        if (weight > 15) {
                double excessWeight = weight - 15;

                baggageCharge = baggageCharge + excessWeight * extraBagFeePerKg;
        }

        totalFare = baseFare + seatSelectionFee + baggageCharge;
        return totalFare;
}

void printBoardingPass() override
{
        cout << "\n[BOARDING PASS - ECONOMY CLASS]\n";

        cout << "Passenger      : "
                << passengerName << endl;

        cout << "Flight Number  : "
                << flightNumber << endl;

        cout << "Base Fare      : $"
                << baseFare << endl;

        cout << "Seat Surcharge : $"
                << seatSelectionFee << endl;

        cout << "Baggage Charge : $"
                << baggageCharge << endl;

        cout << "TOTAL FARE     : $"
                << totalFare << endl;
```

```
		cout << "----------------------------------------\n";
	}


};


#pragma once
#include"FlightTicket.h"

class BusinessTicket :public FlightTicket
{
private:
	double loungeAccessFee;
	double priorityBoardingFee;

	int baggageCount;
	int totalWeightKg;

	double baggageCharge;
	double totalFare;

public:
	BusinessTicket(string passengerName,
		string flightNumber,
		double baseFare, int baggageCount, double totalWeightKg, double
loungeAccessFee, double priorityBoardingFee)
		:FlightTicket(passengerName, flightNumber, baseFare, baggageCount,
totalWeightKg)
	{
		this->baggageCount = baggageCount;
		this->totalWeightKg = totalWeightKg;
		this->loungeAccessFee = loungeAccessFee;
		this->priorityBoardingFee = priorityBoardingFee;

		this->baggageCharge = 0;
		this->totalFare = 0;
	}

	double calculateTotalFare() override {
```

```
                baggageCharge = 0;

                double weight = getTotalWeightKg();

                if (weight > 32) {
                        double excessWeight = weight - 32;

                        baggageCharge = excessWeight * 20;
                }

                totalFare = baseFare + loungeAccessFee + priorityBoardingFee +
baggageCharge;
                return totalFare;
        }

        void printBoardingPass() override
        {
                cout << "\n[BOARDING PASS - BUSINESS CLASS]\n";

                cout << "Passenger      : "
                        << passengerName << endl;

                cout << "Flight Number  : "
                        << flightNumber << endl;

                cout << "Base Fare      : $"
                        << baseFare << endl;

                cout << "lounge fee : $"
                        << loungeAccessFee<< endl;

                cout << "boarding fee : $"
                        << priorityBoardingFee << endl;

                cout << "Baggage Charge : $"
                        << baggageCharge << endl;

                cout << "TOTAL FARE     : $"
                        << totalFare << endl;
```

```
		cout << "----------------------------------------\n";
	}


};
#pragma once
#include"FlightTicket.h"


class FirstClassTicket :public FlightTicket
{
private:
	double conciergeServiceFee;

	int baggageCount;
	double totalWeightKg;

	double baggageCharge;
	double totalFare;

public:
	FirstClassTicket(string passengerName,
			string flightNumber,
			double baseFare, int baggageCount, double totalWeightKg, double
conciergeServiceFee)
		:FlightTicket(passengerName, flightNumber, baseFare, baggageCount,
totalWeightKg)
	{
		this->baggageCount = baggageCount;
		this->totalWeightKg = totalWeightKg;
		this->conciergeServiceFee = conciergeServiceFee;

		this->baggageCharge = 0;
		this->totalFare = 0;
	}

	double calculateTotalFare() override {
		baggageCharge = 0;

		double weight = getTotalWeightKg();
```

```
            if (weight > 50) {
                double excessWeight = weight - 50;

                baggageCharge = excessWeight * 20;
            }

            totalFare = baseFare + conciergeServiceFee + baggageCharge;
            return totalFare;
        }

        void printBoardingPass() override
        {
            cout << "\n[BOARDING PASS - FIRST CLASS]\n";

            cout << "Passenger      : "
                 << passengerName << endl;

            cout << "Flight Number  : "
                 << flightNumber << endl;

            cout << "Base Fare      : $"
                 << baseFare << endl;

            cout << "Concierge Service Fee : $"
                 << conciergeServiceFee << endl;

            cout << "Baggage Charge : $"
                 << baggageCharge << endl;

            cout << "TOTAL FARE     : $"
                 << totalFare << endl;

            cout << "---------------------------------------\n";
        }


};

#include"FlightTicket.h"
#include"EconomyTicket.h"
```

```
#include"BusinessTicket.h"
#include"FirstClass.h"

class PrintoScreen
{
public:
	static void printBoardingPass(FlightTicket* eptr)
	{
		double fare = eptr->calculateTotalFare();
		eptr->printBoardingPass();
		cout << "\nThe calculated Total Fare: $" << fare << endl;
	}
};

int main() {

	string passengerName;
	string flightNumber;
	double baseFare;
	double extraBagFeePerKg;
	double seatSelectionFee;
	int baggageCount;
	double totalWeightKg;
	double baggageCharge;
	double totalFare;
	double loungeAccessFee;
	double priorityBoardingFee;
	double conciergeServiceFee;

	int N;
	cout << "Enter the number of passenger:";
	cin >> N;

	FlightTicket** ticket = new FlightTicket * [N]();

	double totalRevenue = 0;

	for (int i = 0;i < N;i++) {
		cout << "Passenger " << i << "Entry";
```

```
int category;

cout << "\n1.Economy Ticket\n2.Business Ticket\n3.FirstClass Ticket";
cin >> category;


cout << "Enter the Name:";
cin.ignore();
getline(cin, passengerName);
cout << "Enter the flight Number:";
getline(cin, flightNumber);
cout << "Enter the base Fare:";
cin >> baseFare;;
cout << "Enter the baggage Count:";
cin >> baggageCount;
cout << "Enter the total Weight Kg:";
cin >> totalWeightKg;
cin.ignore();



if (category == 1) {
        cout << "Economy Ticket" << endl;



        cout << "Enter the extra bag fee rate";
        cin>> extraBagFeePerKg;

        cout << "Enter the seat Selection Fee";
        cin >> seatSelectionFee;

        ticket[i] = new EconomyTicket(
                passengerName,
                flightNumber,
                baseFare,
                baggageCount,
                totalWeightKg,
                extraBagFeePerKg,
                seatSelectionFee
```

```
        );
    }

    else if (category==2){
        cout << "Business Ticket" << endl;

        cout << "Enter the lounge Access Fee";
        cin >> loungeAccessFee;

        cout << "Enter the priority Boarding Fee";
        cin >> priorityBoardingFee;

        ticket[i] = new BusinessTicket(
            passengerName,
            flightNumber,
            baseFare,
            baggageCount,
            totalWeightKg,
            loungeAccessFee,
            priorityBoardingFee
        );

    }
    else if (category == 3) {
        cout << "First Class Ticket" << endl;



        cout << "Enter the  concierge Service Fee";
        cin >> conciergeServiceFee;

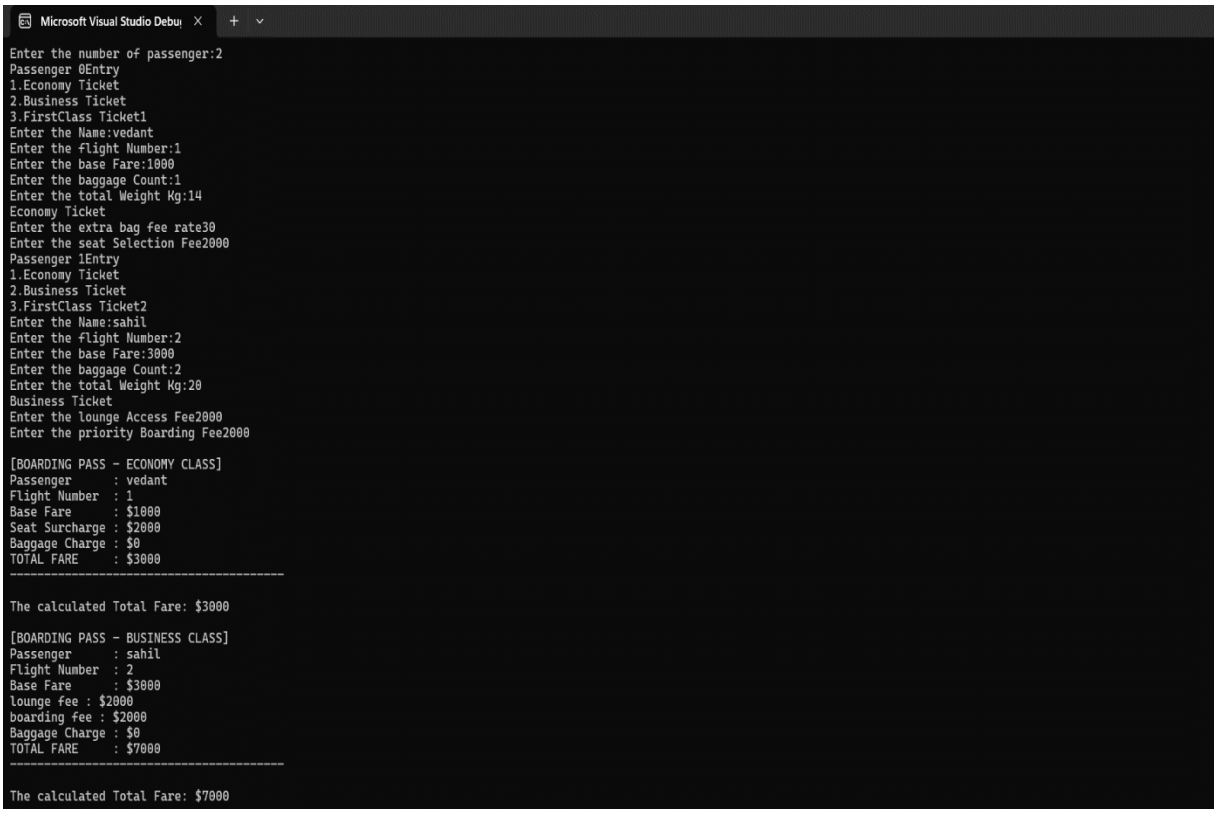
        ticket[i] = new FirstClassTicket(
            passengerName,
            flightNumber,
            baseFare,
            baggageCount,
            totalWeightKg,
            conciergeServiceFee
        );
    }
```

```
        else {
                cout << "Invalid Choice";
                ticket[i] = nullptr;
        }
}

        for (int i = 0; i < N; i++) {
                if (ticket[i] != nullptr) {
                        PrintoScreen::printBoardingPass(ticket[i]);
                        delete ticket[i];
                }
        }

        delete[] ticket;

        return 0;
}
```


Microsoft Visual Studio Debu
Enter the number of passenger:2
Passenger 0Entry
1.Economy Ticket
2.Business Ticket
3.FirstClass Ticket1
Enter the Name:vedant
Enter the flight Number:1
Enter the base Fare:1000
Enter the baggage Count:1
Enter the total Weight Kg:14
Economy Ticket
Enter the extra bag fee rate30
Enter the seat Selection Fee2000
Passenger 1Entry
1.Economy Ticket
2.Business Ticket
3.FirstClass Ticket2
Enter the Name:sahil
Enter the flight Number:2
Enter the base Fare:3000
Enter the baggage Count:2
Enter the total Weight Kg:20
Business Ticket
Enter the lounge Access Fee2000
Enter the priority Boarding Fee2000
[BOARDING PASS - ECONOMY CLASS]
Passenger : vedant
Flight Number : 1
Base Fare : $1000
Seat Surcharge : $2000
Baggage Charge : $0
TOTAL FARE : $3000
The calculated Total Fare: $3000
[BOARDING PASS - BUSINESS CLASS]
Passenger : sahil
Flight Number : 2
Base Fare : $3000
lounge fee : $2000
boarding fee : $2000
Baggage Charge : $0
TOTAL FARE : $7000
The calculated Total Fare: $7000

```
/*
2. Automated Investment Portfolio & Yield Analyzer — Abstract Classes & Polymorphism
*/

#pragma once
#include <iostream>
#include <string>
using namespace std;

class InvestmentAsset {
protected:
	string assetTicker;
	string assetName;
	double initialPrincipal;

public:
	InvestmentAsset(
		string ticker,
		string name,
		double principal
	) {
		this->assetTicker = ticker;
		this->assetName = name;
		this->initialPrincipal = principal;
	}

	virtual ~InvestmentAsset()
	{
	}

	virtual double calculateProjectedValue(
		int durationYears
	) = 0;

	virtual double calculateRiskScore() = 0;

	virtual void displayStatement(
		int durationYears
	) = 0;
```

```
        double getInitialPrincipal()
        {
                return initialPrincipal;
        }

        string getTicker()
        {
                return assetTicker;
        }

        string getName()
        {
                return assetName;
        }
};
```

```
#pragma once
#include"InvestmentAsset.h"


class EquityStock : public InvestmentAsset
{
private:


    double annualGrowthRate;
    double dividendYield;
    double beta;

public:


    EquityStock(
        string ticker,
        string name,
        double principal,
        double growthRate,
        double dividend,
        double betaValue
    )
```

```
    : InvestmentAsset(ticker, name, principal)
{
    annualGrowthRate = growthRate;
    dividendYield = dividend;
    beta = betaValue;
}

double calculateProjectedValue(
    int durationYears
) override
{

    double rate =
        1 + annualGrowthRate + dividendYield;

    double projectedValue =
        initialPrincipal *
        pow(rate, durationYears);

    return projectedValue;
}

double calculateRiskScore() override
{


    double risk = beta * 4.5;



    if (risk > 10.0)
    {
        risk = 10.0;
    }

    if (risk < 1.0)
    {
        risk = 1.0;
    }
```

```
    return risk;
}

void displayStatement(
    int durationYears
) override
{

    double projectedValue =
        calculateProjectedValue(durationYears);


    double risk =
        calculateRiskScore();


    double growth =
        ((projectedValue - initialPrincipal)
            / initialPrincipal) * 100;


    cout << "[Equity Stock] "
        << assetTicker
        << " - "
        << assetName << endl;

    cout << " Initial Principal : $"
        << initialPrincipal << endl;

    cout << " Projected ("
        << durationYears
        << " Yrs) : $"
        << projectedValue
        << " (Net Growth: "
        << growth
        << "%)" << endl;

    cout << " Risk Score : "
        << risk
        << " / 10.00"
```

```
            << endl;

        cout << "-----------------------------------------------------------------"
            << endl;
    }
};
```

```
#pragma once
#include"InvestmentAsset.h"

class RealEstateTrust : public InvestmentAsset
{
private:


    double rentalCapRate;
    double propertyAppreciationRate;
    double managementFeeRate;

public:


    RealEstateTrust(
        string ticker,
        string name,
        double principal,
        double rentalRate,
        double appreciationRate,
        double managementRate
    )
        : InvestmentAsset(ticker, name, principal)
    {
        rentalCapRate = rentalRate;
        propertyAppreciationRate = appreciationRate;
        managementFeeRate = managementRate;
    }



    double calculateProjectedValue(
        int durationYears
```

```
) override
{

   double netRate =
      rentalCapRate
      + propertyAppreciationRate
      - managementFeeRate;


   double projectedValue =
      initialPrincipal *
      pow(1 + netRate, durationYears);

   return projectedValue;
}



double calculateRiskScore() override
{

   return 3.5;
}




void displayStatement(
   int durationYears
) override
{
   double projectedValue =
      calculateProjectedValue(durationYears);

   double risk =
      calculateRiskScore();

   double growth =
      ((projectedValue - initialPrincipal)
```

```
            / initialPrincipal) * 100;


        cout << "[Real Estate] "
           << assetTicker
           << " - "
           << assetName << endl;

        cout << " Initial Principal : $"
           << initialPrincipal << endl;

        cout << " Projected ("
           << durationYears
           << " Yrs) : $"
           << projectedValue
           << " (Net Growth: "
           << growth
           << "%)" << endl;

        cout << " Risk Score : "
           << risk
           << " / 10.00"
           << endl;

        cout << "---------------------------------------------------------------------"
           << endl;
    }
};
```

```
#pragma once
#include"InvestmentAsset.h"
#include <cmath>
class CryptoStaking : public InvestmentAsset
{
private:


    double stakingAPY;
    double networkSlashingPenaltyRate;

public:
```

```
CryptoStaking(
    string ticker,
    string name,
    double principal,
    double apy,
    double penaltyRate
)
    : InvestmentAsset(ticker, name, principal)
{
    stakingAPY = apy;
    networkSlashingPenaltyRate = penaltyRate;
}




double calculateProjectedValue(
    int durationYears
) override
{


    double effectiveYield =
        stakingAPY
        - networkSlashingPenaltyRate;




    double projectedValue =
        initialPrincipal *
        pow(1 + effectiveYield, durationYears);

    return projectedValue;
}

double calculateRiskScore() override
{
```

```
    return 8.5;
}



void displayStatement(
    int durationYears
) override
{
    double projectedValue =
        calculateProjectedValue(durationYears);

    double risk =
        calculateRiskScore();

    double growth =
        ((projectedValue - initialPrincipal)
            / initialPrincipal) * 100;


    cout << "[Crypto Staking] "
        << assetTicker
        << " - "
        << assetName << endl;

    cout << " Initial Principal : $"
        << initialPrincipal << endl;

    cout << " Projected ("
        << durationYears
        << " Yrs) : $"
        << projectedValue
        << " (Net Growth: "
        << growth
        << "%)" << endl;

    cout << " Risk Score : "
        << risk
```

```
            << " / 10.00"
            << endl;

        cout << "------------------------------------------------------------------"
            << endl;
    }
};
```

Code- Source(1).cpp

```
#include <iostream>
#include <string>
#include <cmath>
#include "InvestmentAsset.h"
#include "EquityStock.h"
#include "RealEstateTrtust.h"
#include "CryptoStaking.h"

using namespace std;

int main()
{
    cout << "=== AUTOMATED INVESTMENT PORTFOLIO & YIELD ANALYZER ==="
        << endl;


    int N;
    int durationYears;

    cout << "Enter number of assets (N): ";
    cin >> N;

    cout << "Enter investment duration horizon (years): ";
    cin >> durationYears;

    InvestmentAsset** portfolio =
        new InvestmentAsset * [N]();

    double totalInitialCapital = 0;
    double totalProjectedValue = 0;

    double weightedRiskTotal = 0;
```

```
for (int i = 0; i < N; i++)
{
    cout << "\n--- Asset #"
         << i + 1
         << " Entry ---"
         << endl;


    int type;

    cout << "Select Type "
         << "(1: Equity Stock, "
         << "2: REIT, "
         << "3: Crypto Staking): ";

    cin >> type;


    string ticker;
    string name;
    double principal;

    cout << "Enter Ticker: ";
    cin >> ticker;

    cout << "Enter Asset Name: ";
    getline(cin >> ws, name);

    cout << "Enter Initial Principal ($): ";
    cin >> principal;


    if (type == 1)
    {
        double growthRate;
        double dividend;
        double beta;
```

```
    cout << "Enter Expected Annual Growth Rate "
       << "(e.g. 0.10 for 10%): ";
    cin >> growthRate;

    cout << "Enter Annual Dividend Yield "
       << "(e.g. 0.015 for 1.5%): ";
    cin >> dividend;

    cout << "Enter Beta: ";
    cin >> beta;



    if (principal <= 0) {
       cout << "Invalid principal value, must be > 0. Setting to 0.0\n";
    }


    portfolio[i] =
       new EquityStock(
          ticker,
          name,
          principal,
          growthRate,
          dividend,
          beta
       );
}

else if (type == 2)
{
    double rentalRate;
    double appreciationRate;
    double managementRate;

    cout << "Enter Rental Cap Rate: ";
    cin >> rentalRate;

    cout << "Enter Property Appreciation Rate: ";
    cin >> appreciationRate;
```

```
    cout << "Enter Management Fee Rate: ";
    cin >> managementRate;



    portfolio[i] =
        new RealEstateTrust(
            ticker,
            name,
            principal,
            rentalRate,
            appreciationRate,
            managementRate
        );
}


else if (type == 3)
{
    double apy;
    double penaltyRate;

    cout << "Enter Staking APY: ";
    cin >> apy;

    cout << "Enter Network Slashing Risk Penalty: ";
    cin >> penaltyRate;



    portfolio[i] =
        new CryptoStaking(
            ticker,
            name,
            principal,
            apy,
            penaltyRate
        );
}
```

```
        else
        {
            cout << "Invalid asset type!" << endl;


            portfolio[i] = nullptr;
        }
    }



    cout << "\n\n"
        << "====================== PORTFOLIO PERFORMANCE REPORT
====================="
        << endl;


    for (int i = 0; i < N; i++)
    {
        if (portfolio[i] != nullptr)
        {

            portfolio[i]->displayStatement(durationYears);

            double projectedValue =
                portfolio[i]->calculateProjectedValue(
                    durationYears
                );

            double risk =
                portfolio[i]->calculateRiskScore();


            double principal =
                portfolio[i]->getInitialPrincipal();


            totalInitialCapital =
```

```
            totalInitialCapital + principal;


        totalProjectedValue =
            totalProjectedValue + projectedValue;


        weightedRiskTotal =
            weightedRiskTotal +
            (principal * risk);
    }
}


double totalProfit =
    totalProjectedValue -
    totalInitialCapital;


double totalGrowth =
    (totalProfit / totalInitialCapital) * 100;


double weightedAverageRisk =
    weightedRiskTotal /
    totalInitialCapital;


cout <<
"\n=============================================================================
==="
    << endl;

cout << "PORTFOLIO AGGREGATE SUMMARY:"
    << endl;

cout << " Total Invested Principal : $"
    << totalInitialCapital
    << endl;
```

```
    cout << " Total Projected Value : $"
       << totalProjectedValue
       << endl;

    cout << " Total Projected Profit : $"
       << totalProfit
       << " ("
       << totalGrowth
       << "%)"
       << endl;

    cout << " Weighted Avg Risk Score : "
       << weightedAverageRisk
       << " / 10.00"
       << endl;

    cout <<
"================================================================================
=="
       << endl;


    for (int i = 0; i < N; i++)
    {
       delete portfolio[i];
    }

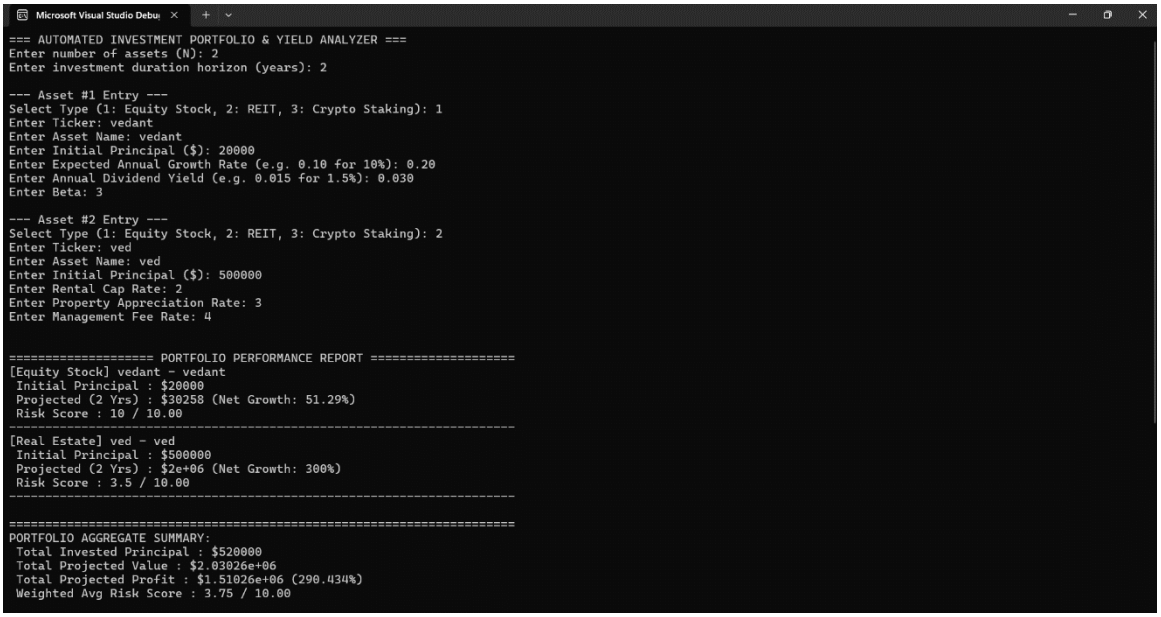
    delete[] portfolio;


    portfolio = nullptr;


    cout << "\nMemory cleaned up successfully. "
       << N
       << " asset instances deallocated."
       << endl;


    return 0;
```

}


Microsoft Visual Studio Debu
=== AUTOMATED INVESTMENT PORTFOLIO & YIELD ANALYZER ===
Enter number of assets (N): 2
Enter investment duration horizon (years): 2
--- Asset #1 Entry ---
Select Type (1: Equity Stock, 2: REIT, 3: Crypto Staking): 1
Enter Ticker: vedant
Enter Asset Name: vedant
Enter Initial Principal ($): 20000
Enter Expected Annual Growth Rate (e.g. 0.10 for 10%): 0.20
Enter Annual Dividend Yield (e.g. 0.015 for 1.5%): 0.030
Enter Beta: 3
--- Asset #2 Entry ---
Select Type (1: Equity Stock, 2: REIT, 3: Crypto Staking): 2
Enter Ticker: ved
Enter Asset Name: ved
Enter Initial Principal ($): 500000
Enter Rental Cap Rate: 2
Enter Property Appreciation Rate: 3
Enter Management Fee Rate: 4
==================== PORTFOLIO PERFORMANCE REPORT ====================
[Equity Stock] vedant - vedant
Initial Principal : $20000
Projected (2 Yrs) : $30258 (Net Growth: 51.29%)
Risk Score : 10 / 10.00
[Real Estate] ved - ved
Initial Principal : $500000
Projected (2 Yrs) : $2e+06 (Net Growth: 300%)
Risk Score : 3.5 / 10.00
PORTFOLIO AGGREGATE SUMMARY:
Total Invested Principal : $520000
Total Projected Value : $2.03026e+06
Total Projected Profit : $1.51026e+06 (290.434%)
Weighted Avg Risk Score : 3.75 / 10.00

C++ BASIC PROGRAMMING ASSIGNMENT

DAY 8

```
/*
Question 1

C++ Assignment: Custom Exceptions and Stream Operator Overloading

Domain: Banking / Digital Wallet System
*/

#include "Wallet.h"

int main()
{
Wallet w;

int choice;
double amount;

do
{
cout << "\n\nDigital Wallet";
cout << "\n1. Create Wallet";
cout << "\n2. Add Money";
cout << "\n3. Withdraw Money";
cout << "\n4. Display Wallet Details";
cout << "\n5. Exit";

cout << "\nEnter your choice: ";
cin >> choice;

switch (choice)
{
case 1:

try
{
cin >> w;

cout << "\nWallet created successfully.";
cout << w;
```

```
}
catch (InvalidAmountException err)
{
cout << "\nError: " << err.what();
}

break;


case 2:

try
{
cout << "\nEnter amount to add: ";
cin >> amount;

w.addMoney(amount);

cout << "\nMoney added successfully.";
cout << w;
}
catch (InvalidAmountException err)
{
cout << "\nError: " << err.what();
}

break;


case 3:

try
{
cout << "\nEnter amount to withdraw: ";
cin >> amount;

w.withdrawMoney(amount);

cout << "\nMoney withdrawn successfully.";
cout << w;
}
catch (InvalidAmountException err)
{
cout << "\nError: " << err.what();
}
catch (InsufficientBalanceException err)
```

```
{
cout << "\nError: " << err.what();
}

break;


case 4:

cout << w;

break;


case 5:

cout << "\nThank you!";

break;


default:

cout << "\nInvalid choice.";
}

} while (choice != 5);

return 0;
}

#pragma once

#include <exception>
#include <cstring>
using namespace std;

class InsufficientBalanceException : public exception
{
private:
char errMsg[50];

public:

InsufficientBalanceException(const char* errMsg)
{
```

```
strcpy(this->errMsg, errMsg);
}

const char* what() const throw()
{
return errMsg;
}
};

#pragma once

#include <exception>
#include <cstring>
using namespace std;

class InvalidAmountException : public exception
{
private:
char errMsg[50];

public:

InvalidAmountException(const char* errMsg)
{
strcpy(this->errMsg, errMsg);
}

const char* what() const throw()
{
return errMsg;
}
};

#pragma once

#include <iostream>
#include <string>
using namespace std;

#include "InvalidAmountException.h"
#include "InsufficientBalanceException.h"

class Wallet
{
private:
int walletId;
```

```
string holderName;
double balance;

public:

// Default Constructor
Wallet()
{
walletId = 0;
holderName = "NA";
balance = 0;
}

// Parameterized Constructor
Wallet(int walletId, string holderName, double balance)
{
this->walletId = walletId;
this->holderName = holderName;

if (balance < 0)
{
InvalidAmountException ex("Balance cannot be negative");
throw ex;
}

this->balance = balance;
}

// Add Money
void addMoney(double amount)
{
if (amount <= 0)
{
InvalidAmountException ex("Amount must be greater than zero");
throw ex;
}

this->balance = this->balance + amount;
}

// Withdraw Money
void withdrawMoney(double amount)
{
if (amount <= 0)
{
InvalidAmountException ex("Amount must be greater than zero");
```

```
throw ex;
}

if (amount > this->balance)
{
InsufficientBalanceException ex("Insufficient wallet balance");
throw ex;
}

this->balance = this->balance - amount;
}

// Operator <<
friend ostream& operator<<(ostream& o, Wallet& obj);

// Operator >>
friend istream& operator>>(istream& i, Wallet& obj);
};


// Insertion Operator
ostream& operator<<(ostream& o, Wallet& obj)
{
o << "\nWallet Details";
o << "\nWallet ID: " << obj.walletId;
o << "\nHolder Name: " << obj.holderName;
o << "\nBalance: " << obj.balance;
o << endl;

return o;
}


// Extraction Operator
istream& operator>>(istream& i, Wallet& obj)
{
cout << "\nEnter Wallet ID: ";
i >> obj.walletId;

cout << "Enter Holder Name: ";
i >> obj.holderName;

cout << "Enter Balance: ";
i >> obj.balance;

if (obj.balance < 0)
```

```
{
InvalidAmountException ex("Balance cannot be negative");
throw ex;
}

return i;
}
```

```
D:\CDAC\ASSIGNMENTS\College_Assignments\8.CPP>g++ 1.cpp -o myexe

D:\CDAC\ASSIGNMENTS\College_Assignments\8.CPP>myexe.exe


Digital Wallet
1. Create Wallet
2. Add Money
3. Withdraw Money
4. Display Wallet Details
5. Exit
Enter your choice: 1

Enter Wallet ID: 101
Enter Holder Name: asd
Enter Balance: 1000

Wallet created successfully.
Wallet Details
Wallet ID: 101
Holder Name: asd
Balance: 1000


Digital Wallet
1. Create Wallet
2. Add Money
3. Withdraw Money
4. Display Wallet Details
5. Exit
Enter your choice: 2

Enter amount to add: 200

Money added successfully.
Wallet Details
Wallet ID: 101
Holder Name: asd
Balance: 1200


Digital Wallet
1. Create Wallet
2. Add Money
3. Withdraw Money
4. Display Wallet Details
5. Exit
Enter your choice: 3

Enter amount to withdraw: 100

Money withdrawn successfully.
Wallet Details
Wallet ID: 101
Holder Name: asd
Balance: 1100


Digital Wallet
1. Create Wallet
2. Add Money
3. Withdraw Money
4. Display Wallet Details
5. Exit
Enter your choice: 4

Wallet Details
Wallet ID: 101
Holder Name: asd
Balance: 1100


Digital Wallet
1. Create Wallet
2. Add Money
3. Withdraw Money
4. Display Wallet Details
5. Exit
Enter your choice: 5

Thank you!
D:\CDAC\ASSIGNMENTS\College_Assignments\8.CPP>
```

# C++ BASIC PROGRAMMING ASSIGNMENT

## DAY 9

/*

Question 1: Binary Student Database

Design and implement an offline Student Information Records System using binary file persistence ('students.dat'). The system must handle student records without loading the entire dataset into main memory, with menu-driven Add, Display Active, and Search operations.

*/

```
#include <iostream>

using namespace std;

#include "FileIO.h"


int main()
{
   char wish;

   do
   {
      int choice;

      int rollNumber;

      cout << "\n\n================================";
      cout << "\n STUDENT INFORMATION SYSTEM";
      cout << "\n================================";

      cout << "\n1. Add New Student";
      cout << "\n2. Display All Active Records";
      cout << "\n3. Search Student by Roll Number";

      cout << "\nEnter the choice: ";
```

```
cin >> choice;


switch (choice)
{
case 1:

   FileIO::AddStudent();

   break;


case 2:

   FileIO::DisplayStudents();

   break;


case 3:

   cout << "\nEnter Roll Number: ";

   cin >> rollNumber;

   FileIO::SearchStudent(rollNumber);

   break;


default:

   cout << "\nInvalid choice";

   break;
}


cout << "\n\nDo you want to perform more operations (y/n): ";
```

```
        cin >> wish;

    } while (
        wish == 'y'
        ||
        wish == 'Y'
        );


    return 0;
}



 #pragma once

 #include <iostream>
 #include <fstream>

 using namespace std;

 #include "Student.h"

 class FileIO
 {
 public:

     static void AddStudent()
     {
         fstream fs;

         Student s1;

         fs.open(
             "students.dat",
             ios::out | ios::binary | ios::app
         );

         if (!fs)
         {
             cout << "\nUnable to open file";
             return;
         }

         s1.accept();

         fs.write(
```

```
        (char*)&s1,
        sizeof(Student)
    );

    fs.close();

    cout << "\nStudent record added successfully";
}


static void DisplayStudents()
{
    fstream fs;

    Student s1;

    fs.open(
        "students.dat",
        ios::in | ios::binary
    );

    if (!fs)
    {
        cout << "\nUnable to open file";
        return;
    }

    cout << "\n\n========== ACTIVE STUDENTS ==========";

    bool found = false;

    while (
        fs.read(
            (char*)&s1,
            sizeof(Student)
        )
        )
    {
        if (s1.getActive())
        {
            s1.display();

            found = true;
        }
    }

    if (!found)
    {
        cout << "\nNo active student records found";
    }

    fs.close();
```

```
}


static void SearchStudent(int rollNumber)
{
   fstream fs;

   Student s1;

   fs.open(
      "students.dat",
      ios::in | ios::binary
   );

   if (!fs)
   {
      cout << "\nUnable to open file";
      return;
   }

   bool found = false;

   while (
      fs.read(
         (char*)&s1,
         sizeof(Student)
      )
      )
   {
      if (
         s1.getRollNumber() == rollNumber
         &&
         s1.getActive()
         )
      {
         cout << "\n\n========== STUDENT FOUND ==========";

         s1.display();

         found = true;

         break;
      }
   }

   if (!found)
   {
      cout << "\nStudent not found";
   }

   fs.close();
}
```

```
};


#pragma once

#include <iostream>
#include <cstring>

using namespace std;

class Student
{
private:
    int rollNumber;
    char fullName[50];
    char department[20];
    float cgpa;
    bool active;

public:

    void accept()
    {
        cout << "\nEnter Roll Number: ";
        cin >> rollNumber;

        cin.ignore();

        cout << "Enter Full Name: ";
        cin.getline(fullName, 50);

        cout << "Enter Department: ";
        cin.getline(department, 20);

        cout << "Enter CGPA: ";
        cin >> cgpa;

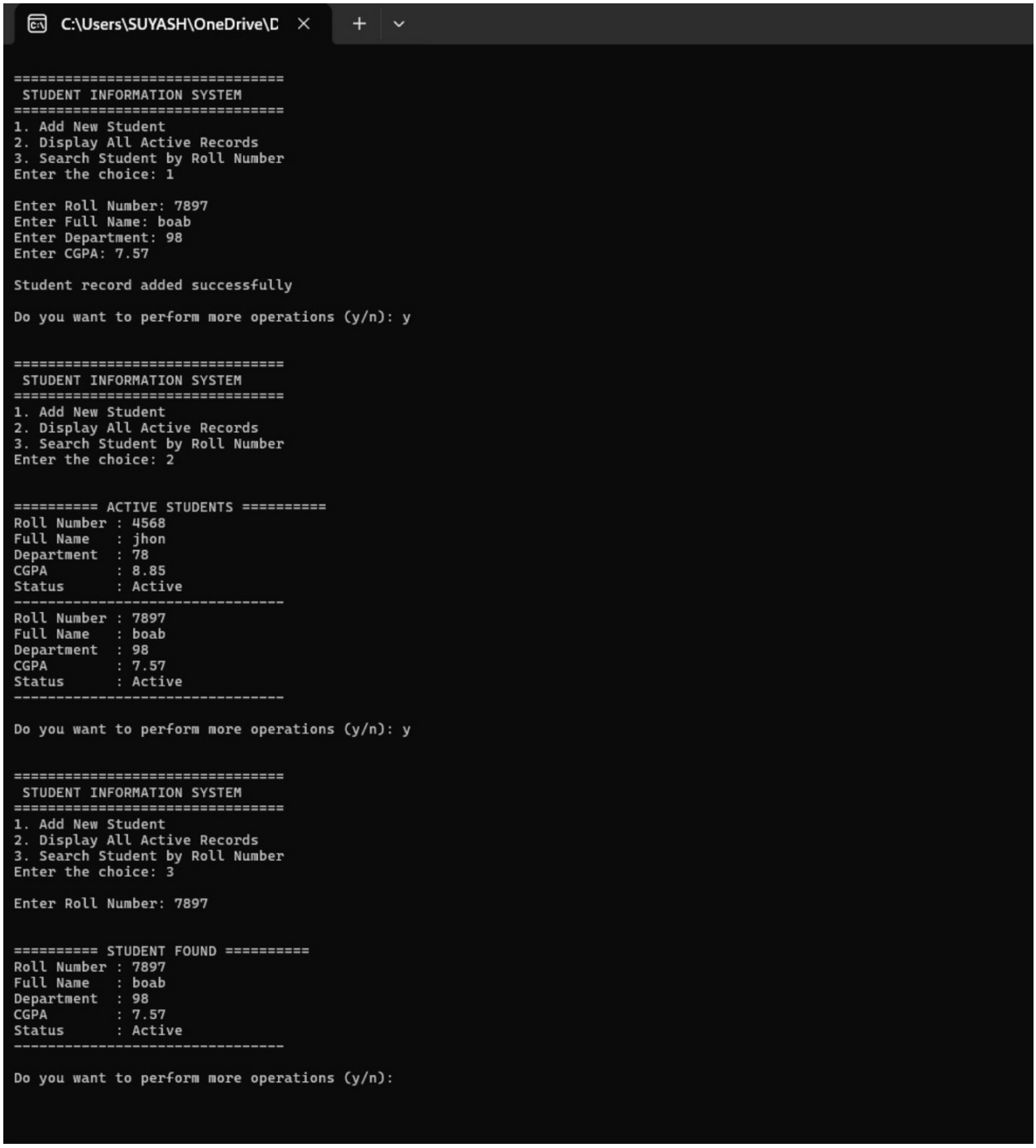
        active = true;
    }

    void display()
    {
        cout << "\nRoll Number : " << rollNumber;
        cout << "\nFull Name   : " << fullName;
        cout << "\nDepartment  : " << department;
        cout << "\nCGPA        : " << cgpa;
        cout << "\nStatus      : "
            << (active ? "Active" : "Inactive");

        cout << "\n--------------------------------";
    }
```

```
    int getRollNumber()
    {
        return rollNumber;
    }

    bool getActive()
    {
        return active;
    }
};
```



```
/*
Question 2: Generic Statistical Pipeline with Template Specialization

Implement generic function templates for minimum, maximum and average calculations, with
string specialization, and test them using int, double and string arrays.
 */

#include "Demo.h"
#include <iostream>

using namespace std;


int main()
{
    // ====================================
    // INTEGER ARRAY
    // ====================================

    int arr1[] = { 45, 12, 89, 3, 27 };

    int min1;
    int max1;

    Demo::findMinMax(
        arr1,
        5,
        min1,
        max1
    );

    cout << "\n================================";
    cout << "\n       INTEGER ARRAY";
    cout << "\n================================";

    cout << "\nArray<int>: [45, 12, 89, 3, 27]";

    cout << "\nMin: " << min1;
```

```
cout << "\nMax: " << max1;

cout << "\nAverage: "
   << Demo::computeAverage(arr1, 5);


  double arr2[] =
{
   3.14,
   2.71,
   9.81,
   1.41
};

double min2;
double max2;

Demo::findMinMax(
   arr2,
   4,
   min2,
   max2
);

cout << "\n\n================================";
cout << "\n       DOUBLE ARRAY";
cout << "\n================================";

cout << "\nArray<double>: "
   << "[3.14, 2.71, 9.81, 1.41]";

cout << "\nMin: " << min2;

cout << "\nMax: " << max2;

cout << "\nAverage: "
   << Demo::computeAverage(arr2, 4);


   string arr3[] =
```

```
{
    "apple",
    "banana",
    "kiwi",
    "strawberry"
};

string shortest;
string longest;

Demo::findMinMax(
    arr3,
    4,
    shortest,
    longest
);

cout << "\n\n================================";
cout << "\n       STRING ARRAY";
cout << "\n================================";

cout << "\nArray<string>: "
    << "[apple, banana, kiwi, strawberry]";

cout << "\nShortest: \""
    << shortest
    << "\" ("
    << shortest.length()
    << ")";

cout << "\nLongest: \""
    << longest
    << "\" ("
    << longest.length()
    << ")";


cout << "\n";

return 0;
```

```
}
```

```
#pragma once

#include <string>

using namespace std;

class Demo
{
public:

    // Generic function to find minimum and maximum
    template <class T>
    static void findMinMax(
        T arr[],
        int size,
        T& minVal,
        T& maxVal)
    {
        minVal = arr[0];
        maxVal = arr[0];

        for (int i = 1; i < size; i++)
        {
            if (arr[i] < minVal)
            {
                minVal = arr[i];
            }

            if (arr[i] > maxVal)
            {
                maxVal = arr[i];
            }
        }
    }


    // Generic function to calculate average
    template <class T>
    static double computeAverage(
        T arr[],
        int size)
    {
        double sum = 0;

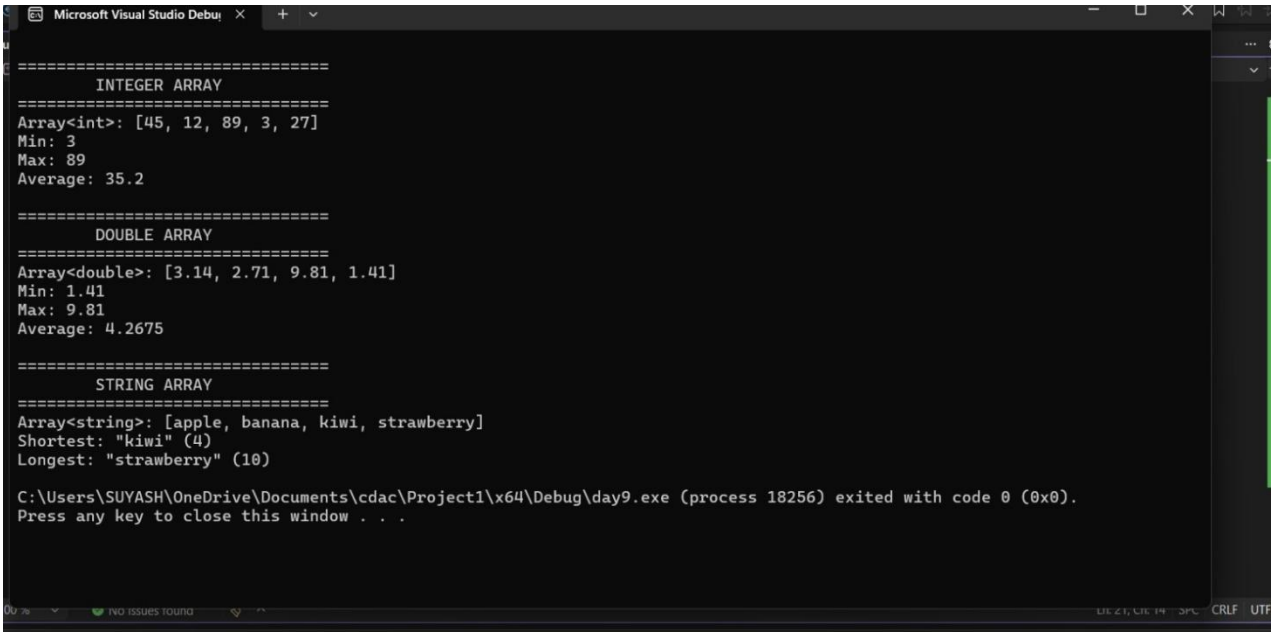
        for (int i = 0; i < size; i++)
        {
            sum = sum + arr[i];
        }
```

```
        return sum / size;
    }


    // Specialization for string
    template <>
    static void findMinMax<string>(
        string arr[],
        int size,
        string& minVal,
        string& maxVal)
    {
        minVal = arr[0];
        maxVal = arr[0];

        for (int i = 1; i < size; i++)
        {
            if (arr[i].length() < minVal.length())
            {
                minVal = arr[i];
            }

            if (arr[i].length() > maxVal.length())
            {
                maxVal = arr[i];
            }
        }
    }
};
```



/*

Question 3: Generic Safe Dynamic Array Container (Custom Vector)

Construct a generic SafeArray<T> container with dynamic resizing, push_back, pop_back, copy constructor, assignment operator, bounds checking, exceptions, and testing with int, string and Product objects.

*/

```
#pragma once

#include<iostream>
#include<string>
#include<stdexcept>

using namespace std;


template<class T>
class SafeArray
{
private:

    T* arr;

    size_t noe;

    size_t capacity;


    void resize()
    {
        size_t newCapacity = capacity * 2;

        T* newArr = new T[newCapacity];


        for (size_t i = 0; i < noe; i++)
        {
            newArr[i] = arr[i];
        }


        delete[] arr;

        arr = newArr;

        capacity = newCapacity;
```

```
    }

public:

      SafeArray()
    {
       this->capacity = 4;

       this->noe = 0;

       this->arr = new T[this->capacity];
    }

    SafeArray(size_t initialSize)
    {
       this->capacity = initialSize;

       this->noe = 0;

       this->arr = new T[this->capacity];
    }

    SafeArray(const SafeArray& other)
    {
       this->capacity = other.capacity;

       this->noe = other.noe;

       this->arr = new T[this->capacity];


       for (size_t i = 0; i < this->noe; i++)
       {
          this->arr[i] = other.arr[i];
       }
    }

    SafeArray& operator=(const SafeArray& other)
    {
       if (this != &other)
       {
          delete[] this->arr;


          this->capacity = other.capacity;

          this->noe = other.noe;

          this->arr = new T[this->capacity];
```

```
        for (size_t i = 0; i < this->noe; i++)
        {
            this->arr[i] = other.arr[i];
        }
    }

    return *this;
}

void push_back(const T& element)
{
    if (this->noe == this->capacity)
    {
        resize();
    }


    this->arr[this->noe] = element;

    this->noe++;
}

void pop_back()
{
    if (this->noe == 0)
    {
        throw out_of_range(
            "Cannot pop from empty array!"
        );
    }

    this->noe--;
}



size_t size()
{
    return this->noe;
}


size_t getCapacity()
{
    return this->capacity;
}


T& operator[](int index)
{
    if (index < 0 ||
```

```
            index >= (int)this->noe)
        {
            throw out_of_range(
                "Index " +
                to_string(index) +
                " out of bounds!"
            );
        }

        return this->arr[index];
    }


    void display()
    {
        for (size_t i = 0; i < this->noe; i++)
        {
            cout << "\n " << this->arr[i];
        }
    }

    ~SafeArray()
    {
        if (this->arr != NULL)
        {
            delete[] this->arr;

            this->arr = NULL;
        }
    }
};


class Product
{
private:

    int id;

    string name;

    double price;


public:

    Product()
```

```
    {
        id = 0;

        name = "";

        price = 0;
    }


    Product(
        int id,
        string name,
        double price)
    {
        this->id = id;

        this->name = name;

        this->price = price;
    }


    void display()
    {
        cout << "\nID    : " << id;

        cout << "\nName  : " << name;

        cout << "\nPrice : $" << price;
    }
};

 #include "SafeArray.h"
 #include<iostream>

 using namespace std;


 int main()
 {
     cout << "\n================================";
     cout << "\n      SafeArray<int>";
```

```
cout << "\n================================";

SafeArray<int> t1;

t1.push_back(10);

t1.push_back(20);

t1.push_back(30);

cout << "\n\nElements:";

t1.display();

cout << "\n\nSize: "
   << t1.size();

cout << "\nCapacity: "
   << t1.getCapacity();

cout << "\n\n================================";
cout << "\n    SafeArray<string>";
cout << "\n================================";

SafeArray<string> t2;

t2.push_back("C++");

t2.push_back("Template");

t2.push_back("Programming");

cout << "\n\nElements:";

t2.display();

cout << "\n\n================================";
cout << "\n    SafeArray<Product>";
cout << "\n================================";

SafeArray<Product> t3;
```

```
cout << "\nSafeArray<Product> Initialized.";


t3.push_back(
    Product(101, "Mouse", 25.50)
);

t3.push_back(
    Product(102, "Keyboard", 50.00)
);

t3.push_back(
    Product(103, "Laptop", 899.99)
);

t3.push_back(
    Product(104, "Monitor", 250.00)
);


cout << "\nPushed 4 items.";

cout << "\nCurrent Capacity: "
    << t3.getCapacity();

t3.push_back(
    Product(105, "Printer", 150.00)
);


cout << "\n\nPushed 5 items.";

cout << "\nDynamic resize triggered.";

cout << "\nCapacity 4 -> "
    << t3.getCapacity();


cout << "\n\nElement at index 2:";

t3[2].display();

t3.pop_back();


cout << "\n\nAfter pop_back:";

cout << "\nSize: "
    << t3.size();
```

```
    cout << "\n\nAttempting access at index 10";


    try
    {
       t3[10].display();
    }
    catch (const out_of_range& e)
    {
       cout << "\nException Caught: "
            << e.what();
    }

    cout << "\n\n================================";
    cout << "\n       Copy Constructor";
    cout << "\n================================";


    SafeArray<int> t4(t1);


    cout << "\nCopied array:";

    t4.display();

    SafeArray<int> t5;

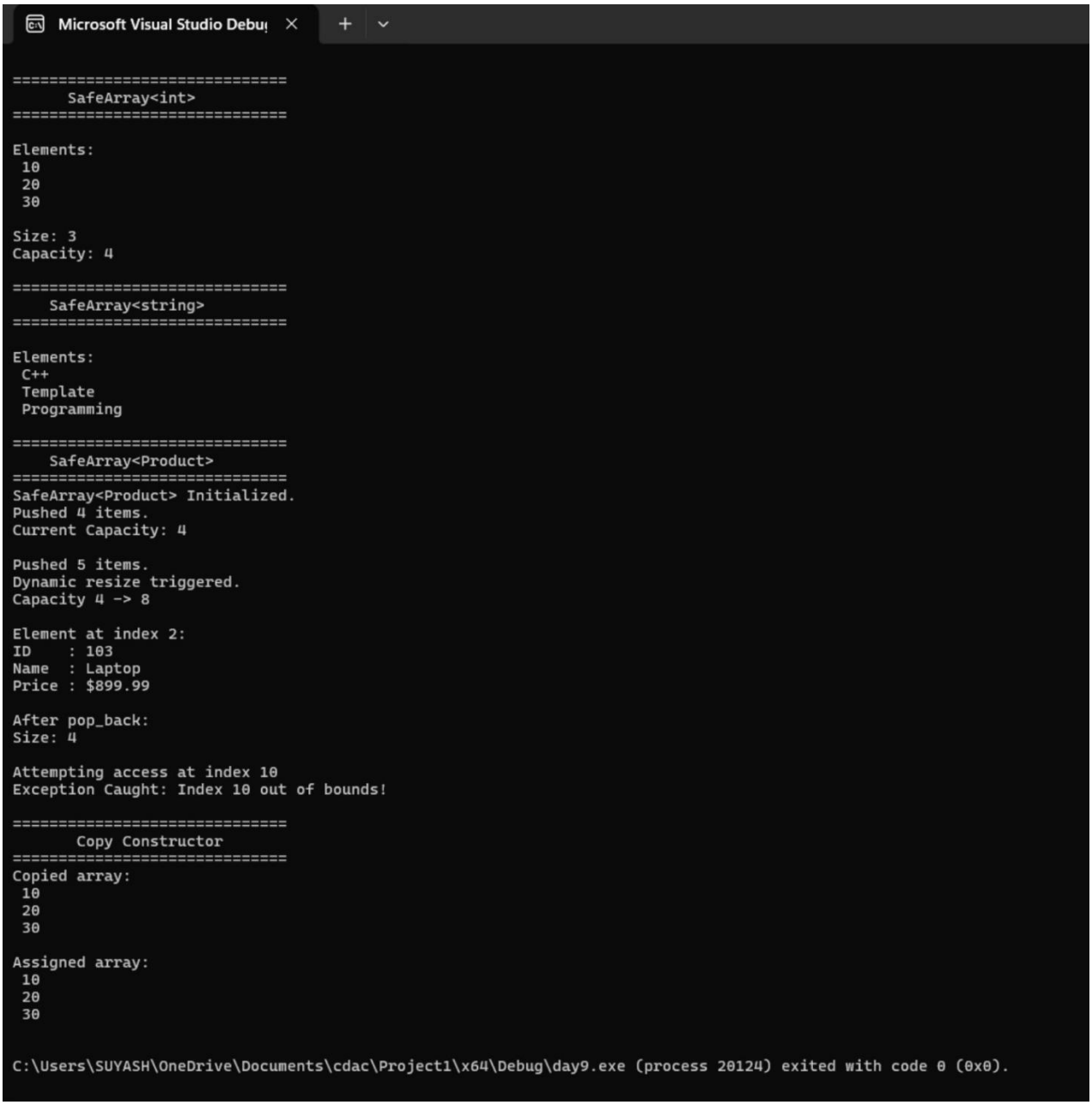
    t5 = t1;


    cout << "\n\nAssigned array:";

    t5.display();


    cout << "\n\n";


    return 0;
}
```


Microsoft Visual Studio Debu
==============================
SafeArray<int>
==============================
Elements:
10
20
30
Size: 3
Capacity: 4
==============================
SafeArray<string>
==============================
Elements:
C++
Template
Programming
==============================
SafeArray<Product>
==============================
SafeArray<Product> Initialized.
Pushed 4 items.
Current Capacity: 4
Pushed 5 items.
Dynamic resize triggered.
Capacity 4 -> 8
Element at index 2:
ID : 103
Name : Laptop
Price : $899.99
After pop_back:
Size: 4
Attempting access at index 10
Exception Caught: Index 10 out of bounds!
==============================
Copy Constructor
==============================
Copied array:
10
20
30
Assigned array:
10
20
30
C:\Users\SUYASH\OneDrive\Documents\cdac\Project1\x64\Debug\day9.exe (process 20124) exited with code 0 (0x0).